

Nombre de la Universidad

Facultad de Ciencias

Título de la Tesis

Tesis presentada para obtener el grado de
Nombre del Grado Académico

Presentado por:

Tu Nombre

Director de Tesis:

Nombre del Director

Ciudad, País

10 de noviembre de 2025

Índice general

1. Introducción	5
2. Gráficas y Superficies	7
2.1. Superficies	7
2.1.1. Conceptos básicos de Topología	8
2.2. Gráficas	16
2.2.1. Gráficas simples y propiedades básicas	16
2.2.2. Representación de gráficas	17
2.2.3. Subgráficas	20
2.2.4. Caminos y ciclos	21
2.2.5. Árboles	22
3. Redes Neuronales Gráficas	27
3.1. Redes Neuronales	27
3.1.1. Conceptos básicos	27
3.1.2. Entrenamiento de redes neuronales	30
3.1.3. Backpropagation	32
3.2. Redes convolucionales CNN	34
3.2.1. Operación de convolución	35
3.2.2. Propiedades de convolución	37
3.2.3. Filtros y mapas de características	38
3.2.4. Arquitectura de una CNN	40
3.3. Redes Neuronales Gráficas	41
3.3.1. Propagación de Mensajes Neuronales	42
3.3.2. Descripción general del mecanismo	43
3.3.3. La GNN fundamental	43
3.3.4. Redes Gráficas Convolucionales (GCNs)	44
4. Construcción de algoritmo de muestreo	47
4.0.1. Superficies estudiadas	47
4.0.2. Métodos de Muestreo	48
4.0.3. Orden por árbol generador mínimo	48
4.0.4. Muestreo por Esferas	50
4.0.5. Reasignación de Adyacencias para Preservar la Estructura	54
4.0.6. Método general para la reducción de nodos	57

4.0.7. Análisis de Complejidad	58
5. Resultados y conclusiones	59
5.0.1. Arquitectura PointNet	59
5.0.2. Resultados de la evaluación	61
5.0.3. Conclusiones	62
A. Definiciones básicas	63

Capítulo 1

Introducción

Capítulo 2

Gráficas y Superficies

La teoría de gráficas y el estudio de superficies son dos ramas de las matemáticas que, a primera vista, podrían parecer independientes. Sin embargo, a lo largo de este capítulo veremos que están profundamente relacionadas a través de la implementación de triangulaciones sobre superficies.

En particular, la representación de superficies mediante gráficas ofrece una nueva perspectiva que permite estudiar propiedades topológicas de los objetos que analizaremos en este trabajo. Una **gráfica** es un conjunto de **vértices** conectados mediante **aristas**, mientras que una **superficie** puede entenderse como un objeto que localmente es homeomorfo al plano $\mathbb{R}^2$, como lo son la esfera o el toro. Al triangular una superficie —definición que abordaremos en este capítulo— es posible inducir una gráfica sobre ella. Esta construcción nos permite aplicar teoremas y algoritmos de la teoría de gráficas al estudio de superficies, facilitando, por ejemplo, su clasificación.

Este capítulo explora la relación entre ambas áreas, enfocándose en cómo la triangulación de superficies se convierte en una herramienta clave para comprender propiedades topológicas fundamentales, así como para implementar algoritmos relevantes en su análisis.

La primera parte, dedicada a definiciones y conceptos topológicos, se basa en los textos de [?] y [?]. La segunda parte, enfocada en definiciones, conceptos y algoritmos relacionados con gráficas y su aplicación en superficies, toma como referencia los textos de [?], [?] y [?], los cuales serán también fundamentales en capítulos posteriores. El capítulo lo presentaremos de una manera rigurosa y formal presentando las definiciones con una estructura predeterminada, además iremos comentando cada definición para que su lectura sea de una forma más comprensible.

2.1. Superficies

Para comprender adecuadamente el objeto de estudio que se construye a lo largo de esta tesis, comenzaremos por introducir uno de los conceptos fundamentales que utilizaremos desde el inicio: el de superficie. Sin embargo, para poder definirlo con rigor, es necesario establecer previamente una serie de conceptos sobre los cuales se apoya esta definición.

Algunas definiciones no se mencionarán en este capítulo ya que podrían considerarse básicas, pero para mantenerlas en contexto se incluyen en el apéndice A.

2.1.1. Conceptos básicos de Topología

Espacio Topológico

En primer lugar definiremos uno de los conceptos que servirán como punto de partida para poder estructurar y poner en contexto los objetos que trataremos a partir de este momento. Un **espacio topológico** lo podemos entender como un conjunto en el que se ha especificado una colección de subconjuntos que cumplen con ciertas propiedades. Estos subconjuntos reciben el nombre de abiertos, los cuales definimos en el apéndice A, y su elección no es arbitraria, sino que debe satisfacer una serie de condiciones que permiten capturar nociones fundamentales como continuidad, convergencia y proximidad, sin necesidad de recurrir a medidas de distancia.

Definición 1. (Espacio topológico)

Sea X un conjunto, una topología en X es una familia $\mathcal{T}$ de subconjuntos abiertos de X , tal que cumple con los siguientes axiomas.

1. Si $\{T_i\}_{i \in \mathcal{I}} \subseteq \mathcal{T}$, $\mathcal{I}$ arbitrario, entonces $\bigcup_{i \in \mathcal{I}} T_i \in \mathcal{T}$.
2. Si $\{T_i\}_{i \in \mathcal{I}} \subseteq \mathcal{T}$, $\mathcal{I}$ finito, entonces $\bigcap_{i \in \mathcal{I}} T_i \in \mathcal{T}$

Por ser $\emptyset$ la intersección de una familia vacía, entonces $\emptyset \in \mathcal{T}$, además, por ser X la unión de una familia vacía, también yace en $\mathcal{T}$. Por lo tanto, $(X, \mathcal{T})$ es un espacio topológico.

A la pareja $(X, \mathcal{T})$ se le denomina **espacio topológico**.

A la familia $\mathcal{T}$ se le denomina **topología** sobre X , y sus elementos se consideran los abiertos del espacio. La presente definición nos proporciona un marco general para trabajar con nociones geométricas de manera más flexible, lo cual resulta útil en contextos que, a simple vista, podrían parecer triviales.

A continuación, presentaremos un ejemplo que, a pesar de su simplicidad aparente, ilustra de una manera adecuada la estructura de un espacio topológico y revela propiedades interesantes.

Ejemplo 1. Sea X un conjunto de dos elementos $X = \{x, y\}$, podemos definir una topología sobre este conjunto de la siguiente forma:

Tomemos los conjuntos abiertos: $\mathcal{T} = \{X, \emptyset, \{x\}\}$

Entonces observamos que:

$$\{x\} \cup X = X, \{x\} \cup \emptyset = \{x\}, X \cup \emptyset = X, \{x\} \cap X = \{x\}, \{x\} \cap \emptyset = \emptyset, X \cap \emptyset = \emptyset$$

Por tanto $(X, \mathcal{T})$ es un espacio topológico, en específico se le conoce como espacio topológico de Sierpiński.

Este espacio, aunque pequeño, nos muestra propiedades útiles en el estudio de conceptos como separación y continuidad.

Su importancia radica en que sirve como el ejemplo más sencillo de un espacio no trivial que distingue entre puntos, y es frecuentemente utilizado como caso de prueba en demostraciones, en la caracterización de funciones continuas, y en demás propiedades.

Veamos que las propiedades de un espacio topológico las podemos restringir hacia subconjuntos más pequeños de este.

Definición 2 (Topología relativa). Sea X un espacio topológico, con topología $\mathcal{T}$. Si Y es un subconjunto de X , entonces la colección:

$$\mathcal{T}_Y = \{Y \cap U \mid U \in \mathcal{T}\}$$

es una topología sobre Y denominada , topología relativa.

Conjuntos cerrados y regiones

Otro de los conceptos en el que nos auxiliaremos para construir y tratar las superficies trabajadas es el concepto de **Conjunto cerrado**, el cual no es más que la contraparte de la definición de conjunto abierto. Notaremos que la relación entre ambos es directa: un conjunto cerrado es aquel cuyo complemento es un conjunto abierto.

Definición 3 (Conjunto cerrado). Sea X un espacio topológico. Decimos que un subconjunto A de X es **cerrado** si el conjunto $X - A$ es abierto.

El siguiente ejemplo es bastante útil visualmente ya que se muestra explícitamente cómo se aplica la definición de conjunto cerrado en un caso concreto.

Ejemplo 2. Sea $A = \{x \times y \mid x \geq 0 \text{ e } y \geq 0\} \subset \mathbb{R}^2$, entonces el conjunto A es cerrado.

Demostración. Observamos que el complemento de A es:

$$\{x \times y \mid x < 0, y \geq 0\} \cup \{x \times y \mid x \geq 0, y < 0\} \cup \{x \times y \mid x < 0, y < 0\}$$

es decir

$$\{x \times y \mid x < 0, y \in \mathbb{R}\} \cup \{x \times y \mid x \in \mathbb{R} \times y < 0\}$$

lo cual es equivalente a

$$(-\infty, 0) \times \mathbb{R} \cup \mathbb{R} \times (0, \infty)$$

, como cada uno de ellos es abierto y la unión de abiertos es abierta, entonces el complemento de A es abierto y por lo tanto A es un conjunto cerrado. $\square$

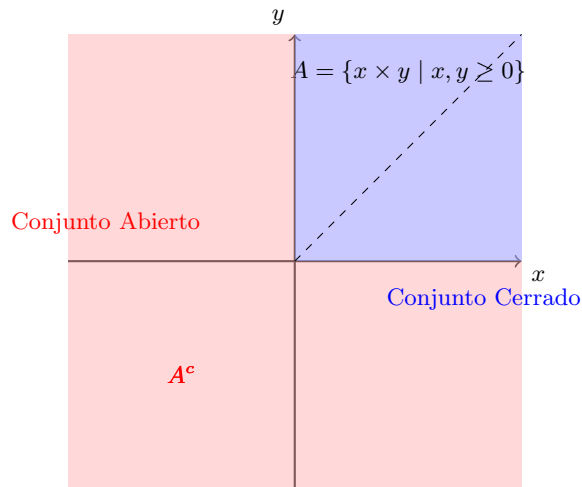


Figura 2.1: Ilustración del suconjunto A

De la mano de las definiciones de conjuntos abierto y cerrados podemos entender que existe el uso de estos conceptos para definir y establecer regiones sobre un espacio topológico. Notaremos que a partir de las definiciones de interior, cerradura y frontera podemos llegar de forma natural a la consideración de un axioma para espacios topológicos, el cual abordaremos más adelante.

Definición 4 (Interior de un conjunto). Sea X un espacio topológico y A un subconjunto de X . El **interior** de A se define como la unión de todos los conjuntos abiertos contenidos en A , es decir

$$\text{Int}A = \bigcup_{i \in \mathcal{I}} A_i \mid A_i \subset A$$

con A_i abierto para $i \in \mathcal{I}$.

Proposición 1. Sea X un espacio topológico y A un conjunto abierto. Entonces A es abierto en X si y sólo si $\text{Int}A = A$

Definición 5 (Cerradura de un conjunto). Sea X un espacio topológico y A un subconjunto de X . La **cerradura** de A se define como la intersección de todos los conjuntos cerrados que contienen a A , es decir

$$\bar{A} = \bigcap_{i \in \mathcal{I}} A_i \mid A \subset A_i$$

con A_i cerrado para $i \in \mathcal{I}$.

Proposición 2. Sea X un espacio topológico y A un conjunto cerrado. Entonces A es cerrado en X si y sólo si $\bar{A} = A$

Definición 6 (Frontera de un conjunto). Sea X un espacio topológico y A un subconjunto de X . Un punto $x \in \bar{A} \cap \overline{X - A}$ se denomina **punto frontera** de A ; al conjunto $\bar{A} \cap \overline{X - A}$ se le llama **frontera** de A .

Espacio de Hausdorff

Notemos que, en cierto modo, las definiciones anteriores nos hablan del comportamiento de los puntos respecto a un conjunto. Con la noción de **interior** podemos describir cuándo un punto “pertenece cómodamente” a la región; la **cerradura** nos indica cuándo un punto está lo suficientemente “cerca” de ella; y la **frontera** señala que un punto está justo “en el límite” de dicho conjunto. A partir de estas ideas surge de forma natural la condición que define un **espacio de Hausdorff**: que dos puntos distintos “no se tocan”, es decir, que existen entornos abiertos disjuntos alrededor de cada uno, cuyas cerraduras tampoco se intersecan. En consecuencia, sus interiores quedan totalmente separados.

Este requisito cobra especial relevancia al pasar de ejemplos familiares, como $\mathbb{R}$ o $\mathbb{R}^2$, donde cada conjunto puntual $\{x_0\}$ es automáticamente cerrado, a topologías más generales en las que ese hecho ya no se cumple. Para evitar estudiar espacios en los que los puntos no puedan separarse nítidamente —espacios que a menudo resultan menos útiles para nuestras intenciones— introducimos la definición de espacios de Hausdorff. En el contexto de esta tesis, este criterio nos permitirá centrar nuestro análisis exclusivamente en objetos que satisfagan dicha condición de separación.

Definición 7. (*Espacio de Hausdorff*)

Sea $(X, \mathcal{A})$ un espacio topológico. Se dice que X es un **Espacio de Hausdorff**, si para cada par x_1, x_2 de puntos distintos de X , existen vecindades U_1, U_2 de x_1, x_2 respectivamente, que son ajenas. Es decir $U_1 \cap U_2 = \emptyset$

Ejemplo 3. Sea $X = \mathbb{R}$ y definamos la topología dada por el conjunto $\mathcal{B} = \{(a, b) \mid 0 \in (a, b) \subset \mathbb{R}\}$.

Tomemos dos puntos distintos arbitrarios, es decir $x \neq y \in X$, entonces queremos ver si existen dos vecindades U, V de x, y tales que $U \cap V = \emptyset$. Pero por la construcción de $\mathcal{B}$, toda vecindad contiene al punto 0, por tanto $U \cap V \neq \emptyset$ y por lo tanto no es de Hausdorff.

Teorema 1. Sea X un espacio topológico de Hausdorff. Sea U un conjunto con un número finito de puntos. Entonces U es cerrado.

Demostración. Sean x, x_0 en X tales que $x \neq x_0$. Como X es de Hausdorff entonces existen vecindades U, V tales que $U \cap V = \emptyset$.

Cómo U no interseca al conjunto x_0 entonces el punto x no puede pertenecer a la cerradura del conjunto x_0 . Como consecuencia $\overline{\{x_0\}} = x_0$, por lo tanto es cerrado. $\square$

Compacidad

Otra de las propiedades importantes que necesitamos para estudiar los objetos centrales, es el concepto de compacidad. De manera intuitiva podemos decir que un espacio es compacto si podemos "cubrirlo" por completo por conjuntos más pequeños pero que en la suma, den el espacio total. Comencemos por definir en primera instancia a que nos referimos por "Cubierta", en específico por "Cubierta abierta".

Definición 8. (*Cubierta abierta*)

Sea X un espacio topológico, sea $\mathcal{A} = \{\mathcal{A}_i\}_{i \in \mathcal{I}}$ una familia de subconjuntos abiertos de X . Se dice que $\mathcal{A}$ es una **cubierta abierta** de X si $X \subseteq \bigcup_{i \in \mathcal{I}} \{\mathcal{A}_i\}$

Definición 9. (*Subcubierta finita*) Sea X un espacio topológico, sea $\mathcal{A}$ una cubierta abierta de X .

Decimos que $\mathcal{A}'$ es una subcubierta de $\mathcal{A}$ si $\mathcal{A}' \subset \mathcal{A}$, en particular si $\mathcal{A}'$ es finito, entonces es una **subcubierta finita**.

Por último buscamos cubrir en su totalidad de manera controlada y finita por estos objetos que definimos anteriormente, de esta forma la definición de espacio compacto queda como sigue:

Definición 10. (*Espacio Compacto*)

Sea X un espacio topológico, se dice que X es **compacto** si de cada cubierta abierta podemos extraer una subcubierta finita de X .

Ejemplo 4. Sea $X = \{0\} \cup \{\frac{1}{n} \mid n \in \mathbb{Z}_+\}$ un subespacio de $\mathbb{R}$.

Demostraremos a continuación que el espacio X es un espacio compacto:

Demostración. Sea $\{\mathcal{A}_\alpha\}$ una cubierta abierta de X , es decir $X \subseteq \bigcup_{\alpha} \mathcal{A}_\alpha$.

Como $0 \in X$ y $\{\mathcal{A}_\alpha\}$ es cubierta abierta, entonces existe un entorno abierto $\mathcal{A}_0$ que contiene

a 0, es decir, existe $\epsilon > 0$ tal que $(-\epsilon, \epsilon) \subseteq \mathcal{A}_0$. Notamos que, en particular existe un N suficientemente grande tal que para todo $n > N$, se cumple que $\frac{1}{n} \in (-\epsilon, \epsilon)$, de ahí que los puntos de la forma $\frac{1}{N}, \frac{1}{(N+1)}, \frac{1}{(N+2)}, \dots$ están en $\mathcal{A}_0$, ahora los puntos restantes $1, 2, \dots, \frac{1}{N-1}$ notamos que son finitos y además están en algún $\mathcal{A}_\alpha$. Es decir están en una subcubierta finita.

Por último notamos que la subcubierta finita de X es de la forma $\mathcal{A}_0 \cup \mathcal{A}_1 \cup \dots \cup \mathcal{A}_{N-1}$ por tanto, X es compacto. □

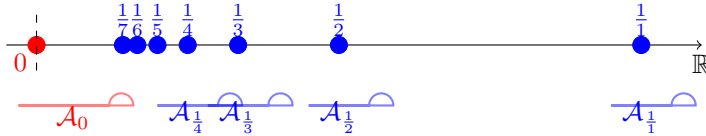


Figura 2.2: Ilustración de la compacidad del espacio X

Lema 1. Sea X un espacio topológico. Sea Y un subespacio de X . Entonces Y es compacto si, y sólo si, cada cubierta de Y por abiertos de X contiene una subcolección finita que cubre a Y .

Demostración. Supongamos primero que Y es compacto y que $\mathcal{A} = \{A_\alpha\}_{\alpha \in J}$ es una cubierta de Y por abiertos de X , entonces la colección:

$$\{A_\alpha \cap Y \mid \alpha \in J\}$$

es una cubierta de Y por conjuntos abiertos en Y ; como Y es compacto entonces existe una subcubierta abierta finita de la forma

$$\{A_{\alpha_1} \cap Y, \dots, A_{\alpha_n} \cap Y\}$$

□

Notamos además que, existe una relación entre los espacios compactos y los espacios de Hausdorff que definimos anteriormente:

Teorema 2. Sea X un espacio topológico de Hausdorff. Sea Y un subespacio compacto de X , entonces Y es cerrado.

Demostración. Sea Y un subespacio compacto de un espacio topológico de Hausdorff X . Queremos mostrar que $X - Y$ es abierto, y por tanto Y será cerrado.

Sea x_0 un punto en $X - Y$. Procederemos a demostrar que existe un entorno de x_0 que no interseca a Y .

Cómo X es de Hausdorff, elegimos entornos disjuntos U_y y V_y de los puntos x_0 y y respectivamente, para cada punto $y \in Y$. Ahora, notamos que la colección

$$\{V_y \mid y \in Y\}$$

es una cubierta de Y por abiertos de X , como Y es compacto, entonces existe una subcolección finita de la forma $V = \{V_{y_1}, \dots, V_{y_n}\}$ que cubre a Y . Además notamos que el conjunto

$$U = \{U_{y_1} \cap \dots \cap U_{y_n}\}$$

formado de tomar las intersecciones de los correspondientes entornos x_0 es disjunto con V . Ya que si $z \in V$ entonces $z \in V_{y_i}$ para algún i , por lo tanto $z \notin U_{y_i}$, y por tanto $z \notin U$. Así U es un entorno de x_0 que no interseca a Y , lo que implica que $X - Y$ es abierto. Por tanto Y es cerrado. $\square$

Espacios numerables

Por último, definiremos otra de las propiedades que nos sirven como base para llegar a definir nuestro objetivo, la cual es la de **numerabilidad**. Esta propiedad nos permite establecer una relación entre los puntos de un espacio topológico y las vecindades que estos poseen. En particular, nos interesa saber si cada punto tiene una base de vecindades numerable, lo que nos permitirá trabajar con espacios más manejables y estructurados.

Definición 11. (*Base de vecindades*)

Sea X un espacio topológico y $x \in X$. Decimos que una familia $\mathcal{B}_x$ de vecindades de x es una **base de vecindades** de x si para cualquier vecindad U de x existe una vecindad $V \in \mathcal{B}_x$ tal que $V \subset U$.

Definición 12. (*Base numerable*) Sea un espacio topológico X . Se dice que X tiene una base numerable en x si existe una base de vecindades numerable.

Definición 13. (*Primer axioma de numerabilidad*) Sea X un espacio topológico. Se dice que X satisface el primer axioma de numerabilidad o que es 1-numerable, si cumple que, cada punto de X tiene una base de vecindades numerable.

Definición 14. (*Segundo axioma de numerabilidad*) Sea X un espacio topológico, se dice que X satisface el segundo axioma de numerabilidad, o que es 2-numerable, si la topología X admite una base numerable.

Ejemplo 5. Observamos que en la topología uniforme $\mathbb{R}^\omega$ satisface el primer axioma de numerabilidad por ser metrizable. Sin embargo no satisface el segundo. Para comprobar este hecho mostraremos que si X es un espacio que tiene base numerable $\mathcal{B}$, entonces cualquier subespacio discreto A de X debe ser numerable. Elijamos para cada $a \in A$ un elemento B_a de la base que interseque a A únicamente en el punto a , si a y b son distintos de A , los conjuntos B_a y B_b son distintos, puesto que el primero contiene a a y el segundo no. Se sigue que la aplicación $a \rightarrow B_a$ es una inyección de A en $\mathcal{B}$, por lo que A debe ser numerable. Ahora bien, el subespacio A de $\mathbb{R}^\omega$ formado por todas las sucesiones de ceros y unos no es numerable y tiene la topología discreta porque $\bar{\rho}(a, b) = 1$ no tiene una base numerable.

Variedades topológicas

Para poder definir una superficie de forma rigurosa, es necesario establecer un contexto adecuado que nos permita trabajar con objetos que cumplan ciertas propiedades topológicas. En este sentido, las **variedades topológicas** son espacios que cumplen con condiciones

específicas, como ser de Hausdorff y 2-numerables, lo que garantiza que cada punto tiene una base de vecindades numerable. Estas condiciones son fundamentales para asegurar que los puntos de la variedad se comporten de manera controlada y predecible, lo cual es esencial para el estudio de superficies.

Definición 15. (*Variedad topológica*) Un espacio topológico X , de Hausdorff, 2-numerable es una **variedad topológica** de dimensión n , también llamada n -variedad, si cada punto $x \in X$ tiene una vecindad homeomorfa a un abierto U en la n -bola cerrada $\mathbb{B}^n$

Definición 16. (*Superficie*)

Una 2 – variedad S es una superficie. Si es compacta y no tiene frontera se le llama **Superficie cerrada**

Ejemplo 6. Sea $S = \{(x, y, z) \in \mathbb{R}^3 | z = x^2 - y^2\}$ Demostraremos que S es una superficie, entonces , tenemos que demostrar que para cada punto $p \in S$ existen conjuntos, $U \subset \mathbb{R}^2$, $W \subset \mathbb{R}^3$ abiertos tal que $p \in W \cap S$ y que existe $h : S \cap W \subset \mathbb{R}^3 \rightarrow U \subset \mathbb{R}^2$ tal que h es un homeomorfismo.

Demostración. Sea $p = (x_0, y_0, z_0) \in S$ entonces por definición $z_0 = x_0^2 - y_0^2$, por otro lado definimos a $W \subset \mathbb{R}^3$ como la bola abierta centrada en p de radio r $\mathcal{B}_p(r)$ y a $U \subset \mathbb{R}^2$ como la vecindad alrededor de (x_0, y_0) , $V(x_0, y_0)$.

Definamos ahora la función $\Pi : S \cap W \rightarrow U$ dada por:

$$\Pi(x, y, x) = (x, y)$$

Es decir la función proyección restringida a $S \cap W$, ahora notemos lo siguiente:

- Π es una función continua ya que es la restricción de la función proyección que es continua en $\mathbb{R}^3$.
- Π es inyectiva pues, dado $u = (x_1, y_1, z_1), w = (x_2, y_2, z_2) \in S \cap W$ tenemos que :

$$\begin{aligned} \Pi(x_1, y_1, z_1) = \Pi(x_2, y_2, z_2) &\Rightarrow \\ (x_1, y_1) = (x_2, y_2) &\Rightarrow \\ x_1 = x_2, y_1 = y_2 & \end{aligned}$$

Por otro lado como $u, w \in S$ entonces:

$$z_1 = x_1^2 - y_1^2 = x_2^2 - y_2^2 = z_2$$

Es decir :

$$u = (x_1, y_1, z_1) = (x_2, y_2, z_2) = w$$

Por tanto Π es inyectiva.

- Sea $(x, y) \in U$ entonces notamos que existe un punto en $p \in S \cap W$ dado por $p = (x, y, x^2 - y^2)$ por tanto Π es sobreyectiva.

- Notamos que Π^{-1} está dada por $\pi^{-1}(x, y) = (x, y, x^2 - y^2)$, como las componentes x, y son continuas y $x^2 - y^2$ es una combinación lineal de componentes continuas entonces $\Pi^{-1}(x, y) = (x, y, x^2 - y^2)$ es continua

Por tanto Π es un homeomorfismo y por tanto $S \cap W$ es homeomorfo a U y por lo tanto S es una superficie. $\square$

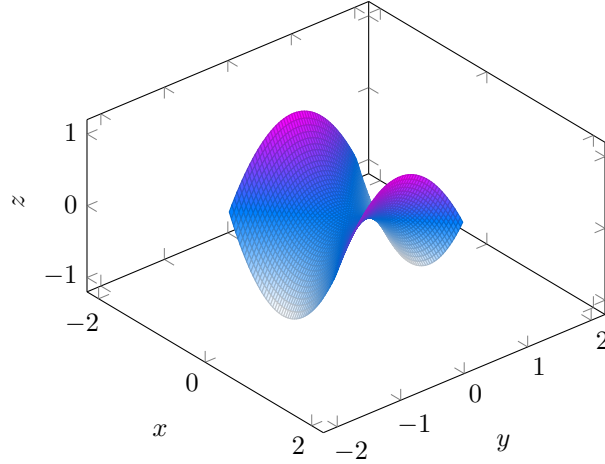


Figura 2.3: Ejemplo de superficie en $\mathbb{R}^3$ comúnmente conocida como "Silla de Montar"

Habiendo establecido una forma rigurosa de definir superficies, es útil explorar cómo podemos representar estas superficies mediante estructuras más manejables y adecuadas para los objetivos de esta tesis. Una técnica clave es la triangulación, que nos permite descomponer una superficie en elementos simples, como triángulos, preservando su estructura topológica. En este contexto, el teorema de triangulación de Radó desempeña un papel central, ya que garantiza que cualquier superficie puede ser triangulada. Este proceso nos permite asociar una gráfica a la superficie al considerar los vértices y aristas de la triangulación, facilitando su estudio como gráfica.

Definición 17 (Triángulo curvo). Sea X un espacio de Hausdorff compacto. Un **triángulo curvo** en X es un par (A, h) formado por un subespacio A de X y un homeomorfismo $h : T \rightarrow A$ donde T es una región triangular cerrada del plano. Si e es una arista de T , entonces $h(e)$ es una arista de A : si v es un vértice de T , entonces $h(v)$ es un vértice de A .

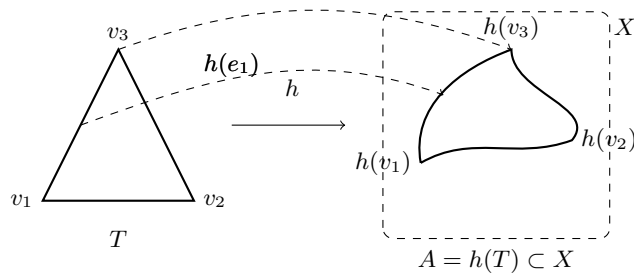


Figura 2.4: Ejemplo de triángulo curvo

Definición 18. Sea X un espacio topológico. Una **triangulación** de una superficie X es una colección de triángulos curvos $A_1, \dots, A_n$ en X cuya unión es X , tal que para $i \neq j$, la intersección $A_i \cup A_j$ es: vacía, un vértice de ambos o una arista de ambos. Si X posee una triangulación, se dirá que X es **triangulable**.

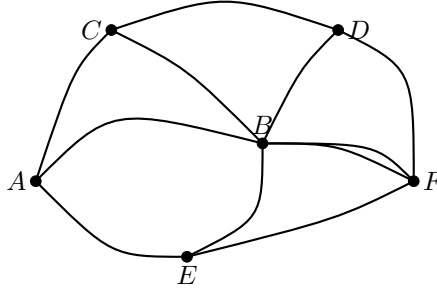


Figura 2.5: Ejemplo de triangulación en una superficie

Teorema 3. (Teorema de Radó)

Toda superficie cerrada S es homeomorfa a una superficie triangulada.

2.2. Gráficas

A partir de las definiciones previas y del Teorema de Radó, es posible introducir nuevos objetos que servirán como base para la caracterización formal de las superficies de interés. De este modo, comenzaremos definiendo las **gráficas**, que son estructuras matemáticas compuestas por vértices y aristas. Estas gráficas nos permitirán representar las superficies de manera más manejable y eficiente, facilitando su estudio y análisis. En particular, al considerar la triangulación de una superficie, cada triángulo curvo puede ser representado como una gráfica, donde los vértices corresponden a los vértices del triángulo y las aristas a sus lados. Así, estableceremos una relación entre superficies y gráficas que facilitará el estudio y análisis eficiente de las superficies consideradas en este trabajo.

2.2.1. Gráficas simples y propiedades básicas

Definición 19 (Gráfica simple). Una gráfica simple G está definida por un par ordenado $G = (V, E)$, donde V es un conjunto cuyos elementos son llamados vértices (o nodos) de la gráfica y E un conjunto cuyos elementos son las aristas de la gráfica.

Regularmente denotamos a V y E como $V(G)$ y $E(G)$ que hacen referencia a la gráfica G , entonces $G = (V(G), E(G))$. Definimos a E como:

$$E(V) = \{\{u, v\} | u, v \in V, u \neq v\}$$

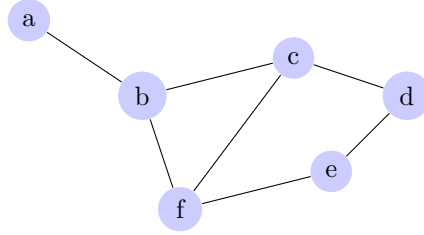
Usualmente denotamos al par $\{u, v\}$ simplemente como uv .

También, observemos las propiedades de las gráficas simples:

Definición 20 (Orden y tamaño). Para una gráfica G , tenemos que:

1. $\nu_G = |V(G)|$ es llamado el orden de G .

2. $\varepsilon_G = |\mathbf{E}(\mathbf{G})|$ es llamado el tamaño de G .
3. Para cada arista $e = uv \in E$ los vértices u y v son llamados extremos de la arista.
4. Los vértices u y v son adyacentes o vecinos si $e = uv \in E$.
5. Dos aristas $e_1 = uv$ y $e_2 = uw$ tienen un extremo común, entonces son adyacentes entre sí.

Figura 2.6: Ejemplo de gráfica simple G

En la figura 2.6 podemos observar que $\nu_G = 6$, $\varepsilon_G = 7$, los extremos de la arista ab son los vértices a y b , también los vértices a y b son vecinos y por último las aristas ab y bf son adyacentes ya que comparten el vértice b .

2.2.2. Representación de gráficas

Matriz de adyacencia

Existen diversas formas de representar gráficas, podemos encontrarlas en su forma visual como la de la figura 2.6, o bien podemos encontrarlas en su forma matricial conocida como **matriz de adyacencia**. La matriz de adyacencia es una representación matricial de una gráfica que permite identificar rápidamente la conexión entre los vértices. En esta matriz, las filas y columnas representan los vértices de la gráfica, y los elementos de la matriz indican si existe una arista entre los vértices correspondientes. A continuación, definiremos formalmente la matriz de adyacencia de una gráfica simple y presentaremos un ejemplo para ilustrar su uso.

Definición 21 (Matriz de Adyacencia). Sea $V_G = \{v_1, \dots, v_n\}$ ordenado. La matriz de adyacencia $\mathbf{A}$ de $\mathbf{G}$ está definida como

$$\mathbf{A}(\mathbf{G}) = [a_{ij}]$$

donde:

$$a_{ij} = \begin{cases} 1 & \text{si } v_i v_j \in \mathbf{E}_G \\ 0 & \text{si no} \end{cases}$$

Ejemplo 7. La matriz de adyacencia de la figura 2.6 quedaría de la siguiente forma:

$$A(G) = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix} \quad (2.1)$$

Notamos que la matriz de adyacencia es una matriz simétrica, ya que si v_i y v_j son vecinos entonces v_j y v_i también lo son, por lo tanto $a_{ij} = a_{ji}$. Además, notamos que la diagonal principal de la matriz de adyacencia es cero, ya que no existe una arista que conecte un vértice consigo mismo en una gráfica simple.

Lista de adyacencia

Otra forma de representar gráficas es mediante la **lista de adyacencia**, que es una representación más compacta y eficiente en términos de espacio. En esta representación, cada vértice se asocia con una lista de sus vecinos, lo que permite identificar rápidamente las conexiones entre los vértices sin necesidad de utilizar una matriz completa.

Definición 22 (Lista de adyacencia). Sea $G = (V, E)$ una gráfica simple. La **lista de adyacencia** de G es un conjunto de listas $\{L_v\}_{v \in V}$ donde cada lista L_v contiene los vértices adyacentes a v . Es decir, para cada vértice $v \in V$, tenemos:

$$L_v = \{u \in V \mid vu \in E\}$$

En la figura 2.6, la lista de adyacencia sería:

Ejemplo 8. Sea $G = (V, E)$ la gráfica de la figura 2.6, entonces la lista de adyacencia de G sería:

Vértice	Lista de adyacencia
a	$\{b\}$
b	$\{a, c, f\}$
c	$\{b, d, f\}$
d	$\{c, e\}$
e	$\{d, f\}$
f	$\{b, c, e\}$

Grado de un vértice

Dada una gráfica $G = (V, E)$, podemos definir el grado de un vértice como el número de aristas que inciden en él. Esto nos permite entender mejor la estructura de la gráfica y las relaciones entre sus vértices. A continuación, definiremos formalmente el vecindario de un vértice y su grado, así como el grado mínimo y máximo de la gráfica.

Definición 23 (Vecindario). Sea $v \in \mathbf{G}$ un vértice de la gráfica $\mathbf{G}$. El **vecindario** de v es el conjunto dado por:

$$N_G(v) = \{u \in G \mid vu \in \mathbf{E}(\mathbf{G})\}$$

El vecindario de un vértice v es el conjunto de todos los vértices que son vecinos de v , es decir, aquellos que están conectados a v por una arista. Este concepto es fundamental para entender la estructura local de la gráfica y las relaciones entre sus vértices.

Definición 24 (Grado de un vértice). *El **grado** de v es el numero de sus vecinos.*

$$d_G(v) = |N_G(v)|.$$

De la definición anterior podemos notar que para cualquier vértice v , podría suceder que v no tenga vecinos o por el otro lado que el resto de los vértices de G sean sus vecinos, recordando que v no puede ser vecino de sí mismo. Así, se tiene que:

$$0 \leq d_G(v) \leq n - 1$$

donde n es el orden la gráfica G .

Definición 25 (Vértice aislado). *Cuando $d_G(v) = 0$ decimos que v es un vértice aislado.*

Dada una gráfica $G = (V, E)$ con $V(G) = \{v_1, \dots, v_n\}$ sabemos que el conjunto:

$$\{d_G(v_1), \dots, d_G(v_n)\}$$

Es un conjunto finito de números naturales pues el conjunto $V(G)$ lo es, entonces de esta forma podemos definir a $\delta(G)$ como el grado mínimo de G y $\Delta(G)$ como el grado máximo de G .

Definición 26 (Grado mínimo y grado máximo). *Sea $G = (V, E)$ una gráfica simple, tenemos que;*

1. $\delta(G) = \min\{d_G(v_1), \dots, d_G(v_n)\}$ es el grado mínimo de G .
2. $\Delta(G) = \max\{d_G(v_1), \dots, d_G(v_n)\}$ es el grado máximo de G .

Para la gráfica de la figura 2.6 tenemos que $\delta(G) = 1$ y que $\Delta(G) = 3$.

Lema 2. (*Lema del saludo de manos*) *Para cada gráfica G tenemos que:*

$$\sum_{v \in V(G)} d_G(v) = 2 \varepsilon_G$$

Además, el número de vértices de grado impar es par.

Demostración. Por definición de grado de un vértice, tenemos que:

$$\sum_{v \in V(G)} d_G(v) = \sum_{v \in V(G)} |N_G(v)|$$

Ahora, notamos que cada arista e de G contribuye al grado de sus dos extremos, es decir, si $e = uv$, entonces $d_G(u) + d_G(v)$ cuenta la arista uv dos veces. Por lo tanto, al sumar los grados de todos los vértices, contamos cada arista dos veces. Así, tenemos que:

$$\sum_{v \in V(G)} d_G(v) = 2 \varepsilon_G$$

Por otro lado, notamos que el número de vértices de grado impar es par, ya que cada arista conecta dos vértices y por lo tanto contribuye a la paridad del grado de ambos extremos. Si tuviéramos un número impar de vértices de grado impar, entonces la suma total de los grados sería impar, lo cual contradice el hecho de que la suma total es igual a $2 \varepsilon_G$, un número par. $\square$

Definición 27 (Grado promedio). *Dada una gráfica G , definimos el **grado promedio** de G como:*

$$d(G) = \frac{1}{\nu_G} \sum_{v \in V(G)} d_G(v)$$

De ahí se reduce inmediatamente por **lema 5** que

$$d(G) = 2 \frac{\varepsilon_G}{\nu_G}$$

Proposición 3. *Dada una gráfica G , tenemos que*

$$\delta(G) \leq d(G) \leq \Delta(G).$$

Demostración. Sabemos que, por definición de grado mínimo y máximo, para todo $v \in V(G)$ se cumple que $\delta(G) \leq d_G(v) \leq \Delta(G)$. Sumando sobre todos los vértices:

$$\sum_{v \in V(G)} \delta(G) \leq \sum_{v \in V(G)} d_G(v) \leq \sum_{v \in V(G)} \Delta(G)$$

Como $\delta(G)$ y $\Delta(G)$ son constantes respecto a la suma, esto se puede escribir como:

$$\nu_G \cdot \delta(G) \leq \sum_{v \in V(G)} d_G(v) \leq \nu_G \cdot \Delta(G)$$

Dividiendo entre ν_G en toda la desigualdad, obtenemos:

$$\delta(G) \leq \frac{1}{\nu_G} \sum_{v \in V(G)} d_G(v) \leq \Delta(G)$$

Por definición, el término central es el grado promedio $d(G)$, así que finalmente:

$$\delta(G) \leq d(G) \leq \Delta(G)$$

$\square$

2.2.3. Subgráficas

Debido a los métodos que se utilizarán más adelante en el estudio de los objetos y debido al gran número de datos a procesar, es necesario introducir conceptos que se pueden emplear para el estudio local de las gráficas. De este modo introduciremos el concepto de subgráfica.

Definición 28 (Subgráfica). *Sea $G = (V, E)$ una gráfica simple. Una subgráfica de G es una gráfica $H = (V_H, E_H)$ tal que: $\subseteq G$, si $V(H) \subseteq V(G)$ Y $E(H) \subseteq E(G)$.*

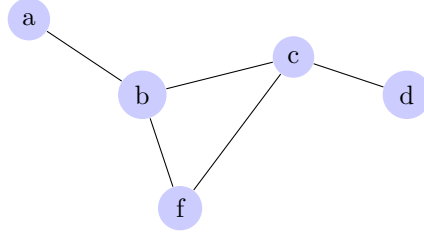


Figura 2.7: Ejemplo de subgráfica de G

Ejemplo 9.

Definición 29. Sea G una gráfica simple y H una subgráfica de G . Decimos que una subgráfica $H \subseteq G$ abarca G (y H es una subgráfica generadora de G) si cada vértice de G está en H , i.e $V(H) = V(G)$

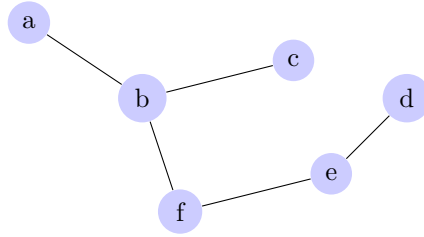


Figura 2.8: Ejemplo de una gráfica H que abarca G

Ejemplo 10.

Definición 30. Sea G una gráfica simple. Decimos que una subgráfica $H \subseteq G$, es una subgráfica inducida, si $E(H) = E(G) \cap E(V_H)$. En este caso H es inducido por el conjunto $V(H)$ de vértices.

En otras palabras, una subgráfica inducida es aquella que contiene todas las aristas de G que conectan los vértices de H . Esto significa que si dos vértices en H están conectados por una arista en G , entonces esa arista también estará presente en H .

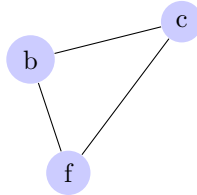


Figura 2.9: Ejemplo de una gráfica inducida

Ejemplo 11.**2.2.4. Caminos y ciclos**

Uno de los conceptos fundamentales que se introducirán y además se implementarán en la creación de los algoritmos de la sección final es el concepto de caminos, este concepto se

utilizará cómo herramienta para el estudio y recorrido de la gráfica completa.

Definición 31 (Camino, ciclo y caminos cerrados). Sea $e_i = u_i u_{i+1} \in E(G)$ aristas de G para $i \in \{1, \dots, k\}$.

Un camino es una gráfica no vacía $W = (V, E)$ de la forma;

$$V = \{u_0, u_1, \dots, u_k, u_{k+1}\} \quad E = \{e_0, e_1, \dots, e_k\}$$

donde: u_i es adyacente a u_{i+1} para todo $i \in \{1, \dots, k\}$.

Además:

1. W es **cerrado**, si $u_0 = u_{k+1}$.
2. W es un **camino**, si $u_i \neq u_j$, para todo $i \neq j$.
3. W es un **ciclo**, si es cerrado y $u_i \neq u_j$ para $i \neq j$ excepto para $u_1 = u_{k+1}$

Además, el número $l(W)$ es el tamaño o longitud del camino y está dado por $l(W) = |E(W)|$

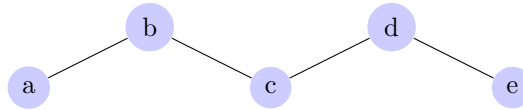


Figura 2.10: Ejemplo de camino

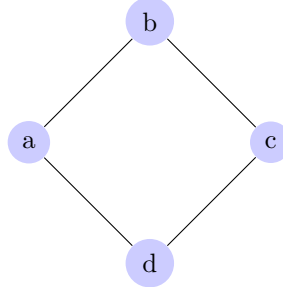


Figura 2.11: Ejemplo de ciclo

Ejemplo 12.

2.2.5. Árboles

Una forma de reducir el número de vértices sin que la gráfica pierda gran parte de su información, es mediante el estudio de los árboles, veremos su definición y los conceptos más importantes relacionados con el estudio de estas estructuras.

Definición 32 (Gráfica conexa). Una gráfica $G = (V, E)$ es conexa si para cualquier par de vértices u y v de G si existe un camino que inicia en u y termina en v

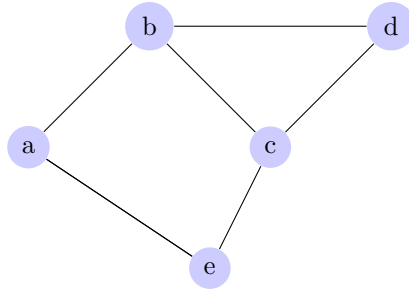


Figura 2.12: Ejemplo de gráfica conexa

Ejemplo 13.

Definición 33 (Componente conexo). Sea $G = (V, E)$ una gráfica. Un **componente conexo** de G es una subgráfica H tal que:

1. H es conexas.
2. H es maximal con respecto a la propiedad de ser conexas. Es decir, no existe una subgráfica conexas más grande que contenga a H .

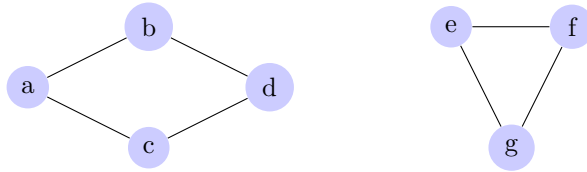


Figura 2.13: Ejemplo de un grafo con dos componentes conexas

Ejemplo 14.

Notamos que una gráfica G es conexas si y sólo si tiene una única componente conexas, es decir, si no se puede dividir en subgráficas conexas más pequeñas. En caso contrario, si G no es conexas, entonces se puede dividir en múltiples componentes conexas, cada una de las cuales es maximal con respecto a la propiedad de ser conexas.

Definición 34 (Puente). Dada un gráfica G conexas y una arista $e \in E_G$. Decimos que e es un **puente** de la gráfica G , si $G - e$ es desconexas.

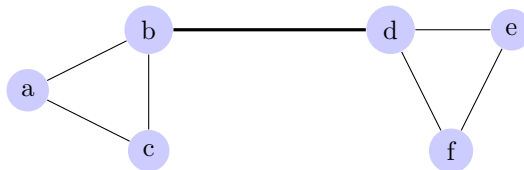


Figura 2.14: Ejemplo de grafo con un puente (arista de corte)

Ejemplo 15.

En la figura 2.14 notamos que la arista de es un puente, ya que si la eliminamos, la gráfica resultante se vuelve desconexas, separando los vértices a, b y c de los vértices d, e y f .

Proposición 4. *Sea una gráfica G . Un vértice $e \in E(G)$ es un puente de G si y sólo si e no está en ningún ciclo de G .*

Demostración. Primero notemos que $e = uv$ es un puente sí y sólo si u, v pertenecen a diferentes componentes conexas de $G - e$, pues de lo contrario implicaría que existe un camino que conecta a u con v lo cual sería una contradicción ya que $G - e$ es desconexo.

($\Rightarrow$) Por contrapositiva, Supongamos que existe un ciclo de G que contiene a e , entonces existe un camino tal que $C = \{u_0 = u, v, \dots, u_{k+1} = u\}$. De aquí, notamos que en $G - e$ existe un camino $C' = \{v_0 = v, \dots, v_{k+1} = u\}$, entonces existe un camino alternativo que conecta a u con v lo cual implica que $G - e$ es conexa. Por tanto $e = uv$ no es puente.

($\Leftarrow$) Supongamos que $e = uv$ no es puente, por la observación hecha al inicio esto implica que u, v pertenecen a la misma componente conexa de $G - e$. Entonces existe un camino que $C = \{v_0 = v, \dots, v_{k+1} = u\}$ conecta a v con u , de ahí notamos que eC es un ciclo pues $eC = \{u, v_0 = v, \dots, v_{k+1} = u\}$. $\square$

Definición 35 (Gráfica acíclica). *Una gráfica es llamada acíclica si no contiene ciclos.*

Definición 36 (Árbol). *Un árbol es una gráfica acíclica y conexa.*

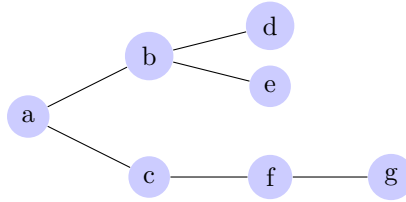


Figura 2.15: Ejemplo de una gráfica acíclica (un árbol)

Ejemplo 16.

De la **Proposición 2.2** y la **Definición 22** se deriva el siguiente corolario.

Corolario 1. *Una gráfica conexa es un árbol si y sólo si todas sus aristas son puentes.*

Notamos que un árbol es una gráfica conexa y acíclica, además de que todas sus aristas son puentes. Esto implica que si eliminamos cualquier arista de un árbol, la gráfica resultante se vuelve desconexa, lo cual es una propiedad clave de los árboles.

Teorema 4. *Los siguientes enunciados son equivalentes:*

1. T es un árbol
2. Cualesquiera dos vértices están conectados en T por un único camino.
3. T es acíclico y $\varepsilon_T = \nu_T - 1$

Demostración. Sea $\nu_T = n$, Si $n=1$ el teorema es trivial, así que supongamos para $n \geq 2$.

(1) $\Rightarrow$ (2) Supongamos que T es un árbol. Sean u y v dos vértices de T y por contradicción supongamos que existen dos caminos P y Q tales que conectan a u y v . Sin pérdida de generalidad supongamos que $l(P) > l(Q)$. Entonces existe al menos una arista

e que está en P pero no está en Q . Por **Corolario 2** sabemos que cada arista de T es un puente. Y por lo visto anteriormente entonces u y v están en diferentes componentes conexas de $T - e$. Pero notamos que por el camino Q siguen estando conectados. Lo cual es una contradicción ya que implicaría que u y v no estén en diferentes componentes conexas.

(2) $\Rightarrow$ (3) Esta implicación la probaremos por inducción sobre n .

Para $n = 2$. Sabemos que no existen componentes disconexas pues cualesquiera dos vértices están conectados por un camino, por lo cual $\varepsilon_T = 1 = 2 - 1 = \nu_T - 1$.

Supongamos que la afirmación se cumple para los valores menores a n y probemos para n . Sea P el máximo camino en T entre u y v , es decir que no existen aristas e tales que eP o Pe sean caminos. De ahí que $d_T(v) = 1$. Si $uw \in E(T)$ entonces $w \in P$. Ahora notamos que la subgráfica $T - v$ está conectada por un único camino P' , además notamos que $l(P)$ es a lo más n y cómo $l(P) > l(P')$ entonces $l(P') \leq n - 1$ dado que la subgráfica $T - v$ está conectada por un único camino (Pues T lo está) y por hipótesis de inducción tenemos que:

$$\varepsilon_{T-v} = \nu_{T-v} - 1 = (n - 1) - 1 = n - 2$$

Y de ahí que:

$$\varepsilon_T = \varepsilon_{T-v} + 1 = n - 1$$

(3) $\Rightarrow$ (1) Supongamos (3). Basta con probar que T es conexo. Sean $T_i = (V_i, E_i)$ componentes conexas de T para $i \in \{1, \dots, k\}$. Sabemos que T es acíclico, por lo tanto sus componentes conexas T_i también lo son por tanto $|E_i| = |V_i| - 1$. Notemos que:

$$\nu_T = \sum_{i=1}^k |V_i|, \varepsilon_T = \sum_{i=1}^k |E_i|$$

Entonces:

$$n - 1 = \varepsilon_T = \sum_{i=1}^k (|V_i| - 1) = \sum_{i=1}^k |V_i| - k = n - k$$

$$\therefore k = 1$$

Lo cual quiere decir que T es un sólo componente conexo, por lo tanto T es un árbol. $\square$

Definición 37 (Árbol generador). *Dada una gráfica conexas G una subgráfica T de G es un subárbol generado de G si satisface que;*

1. T es una subgráfica generadora de G y
2. T es un árbol.

Teorema 5. *Cada gráfica conexas contiene un árbol generador.*

Demostración. Sea $H \subset G$ la mínima gráfica generadora conexas de G , es decir $H - e$ es disconexa para todo $e \in E(H)$. Esta gráfica es obtenida de G removiendo todas las aristas que no son puentes:

1. Hacemos $H_0 = G$.
2. Para $i \geq 0$ sea $H_{i+1} = H_i - e_i$ donde e no es un puente de H_i . Como e_i no es un puente H_{i+1} es una subgráfica generadora y conexa de H_i y por lo tanto de G .
3. De este forma hacemos $H = H_k$ cuando sólo quedan puentes en la subgráfica. Y por el **Corolario 2** H es un árbol.

□

Definición 38 (Árbol generador mínimo). *Un árbol generador mínimo es un árbol generador tal que la suma de los pesos de sus aristas es la mínima posible, es decir.*

$$w(T) = \sum_{(u,v) \in T} \min\{w(u,v)\}$$

Donde $w(u,v)$ es el peso que hay entre el vértice u y la arista v .

Capítulo 3

Redes Neuronales Gráficas

En las últimas décadas, las redes neuronales han emergido como una herramienta fundamental en el campo del aprendizaje automático y la inteligencia artificial. Su capacidad para aprender de grandes volúmenes de datos, reconocer patrones complejos y adaptarse a distintos entornos ha sido clave para su popularidad y adopción en una amplia gama de tareas. Estas aplicaciones abarcan desde el procesamiento del lenguaje natural y la visión por computadora hasta la predicción de series temporales y la toma de decisiones, consolidando a las redes neuronales como una tecnología esencial en desarrollos tan diversos como los sistemas de recomendación y la conducción autónoma.

Aunque la literatura actual ha abordado ampliamente las redes neuronales convencionales, la creciente demanda de resolver problemas más complejos ha impulsado la evolución hacia arquitecturas más especializadas. Entre estas, las Redes Neuronales Gráficas (Graph Neural Networks o GNN) se han destacado por su capacidad para trabajar con datos estructurados en forma de gráficas, lo que las hace especialmente útiles para el objetivo de este trabajo.

Este capítulo ofrecerá una introducción formal a las redes neuronales gráficas y explorará la necesidad de adoptar estas estructuras avanzadas para el estudio de dominios donde la naturaleza de los datos es inherentemente no lineal o interdependiente.

3.1. Redes Neuronales

3.1.1. Conceptos básicos

Una red neuronal puede entenderse como la composición de múltiples funciones no lineales, que en conjunto forman un modelo capaz de aprender representaciones complejas de los datos. Estas redes están inspiradas en la estructura y funcionamiento del cerebro humano, donde las neuronas se conectan entre sí para procesar información y tomar decisiones.

A continuación se dará la definición formal del modelo de red neuronal partiendo de la unidad más básica denominada como perceptrón.

Definición 39 (Perceptrón). Sea $\mathbf{w} \in \mathbb{R}^d$ un vector de pesos, $\mathbf{x} \in \mathbb{R}^d$ un vector de entrada y $b \in \mathbb{R}$ un sesgo (umbral). Definimos la función

$$f : \mathbb{R}^d \rightarrow \{0, 1\}, \quad f(\mathbf{x}) = \chi_{\{\mathbf{y} \in \mathbb{R}^d : \mathbf{w}^T \mathbf{y} + b > 0\}}(\mathbf{x}),$$

donde χ_A denota la función característica del conjunto $A \subseteq \mathbb{R}^d$. En forma explícita,

$$f(\mathbf{x}) = \begin{cases} 1, & \text{si } \mathbf{w}^T \mathbf{x} + b > 0, \\ 0, & \text{si } \mathbf{w}^T \mathbf{x} + b \leq 0. \end{cases}$$

De la anterior definición, podemos observar que el perceptrón es un clasificador lineal que toma una decisión binaria basada en una combinación lineal de las entradas ponderadas por los pesos y ajustada por un sesgo. La función característica χ_A actúa como una función de activación que determina si la salida es 1 o 0, dependiendo de si la combinación lineal supera el umbral definido por el sesgo.

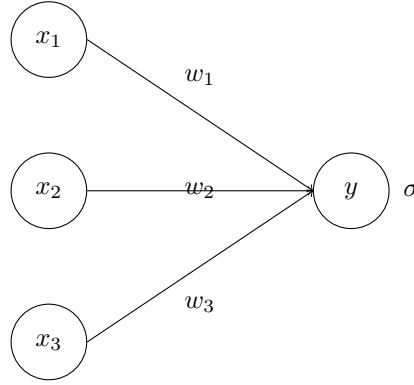


Figura 3.1: Ejemplo de perceptrón

Con el perceptrón definido, podemos construir redes neuronales más complejas al combinar múltiples perceptrones en capas. Una capa, cómo lo veremos a continuación, es la combinación de funciones entre un perceptrón y una función de activación.

Definición 40 (Capa de una red neuronal). *Sea $L \in \mathbb{N}$ el índice de una capa en una red neuronal. Sean: $\mathbf{W}^{(L)} \in \mathbb{R}^{n_L \times n_{L-1}}$ la matriz de pesos que conecta la capa $(L-1)$ con la capa L , $\mathbf{b}^{(L)} \in \mathbb{R}^{n_L}$ el vector de sesgos (bias), $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ una función de activación (aplicada componente a componente).*

Definimos la **capa L -ésima** como la aplicación

$$f^{(L)} : \mathbb{R}^{n_{L-1}} \longrightarrow \mathbb{R}^{n_L},$$

dada por

$$f^{(L)}(\mathbf{x}) = \sigma\left(\mathbf{W}^{(L)}\mathbf{x} + \mathbf{b}^{(L)}\right),$$

donde la función de activación σ se aplica de manera componente a componente sobre el vector $\mathbf{W}^{(L)}\mathbf{x} + \mathbf{b}^{(L)}$.

En particular:

$$f^{(0)}(\mathbf{x}) = \mathbf{x}, \quad \mathbf{x} \in \mathbb{R}^{n_0}.$$

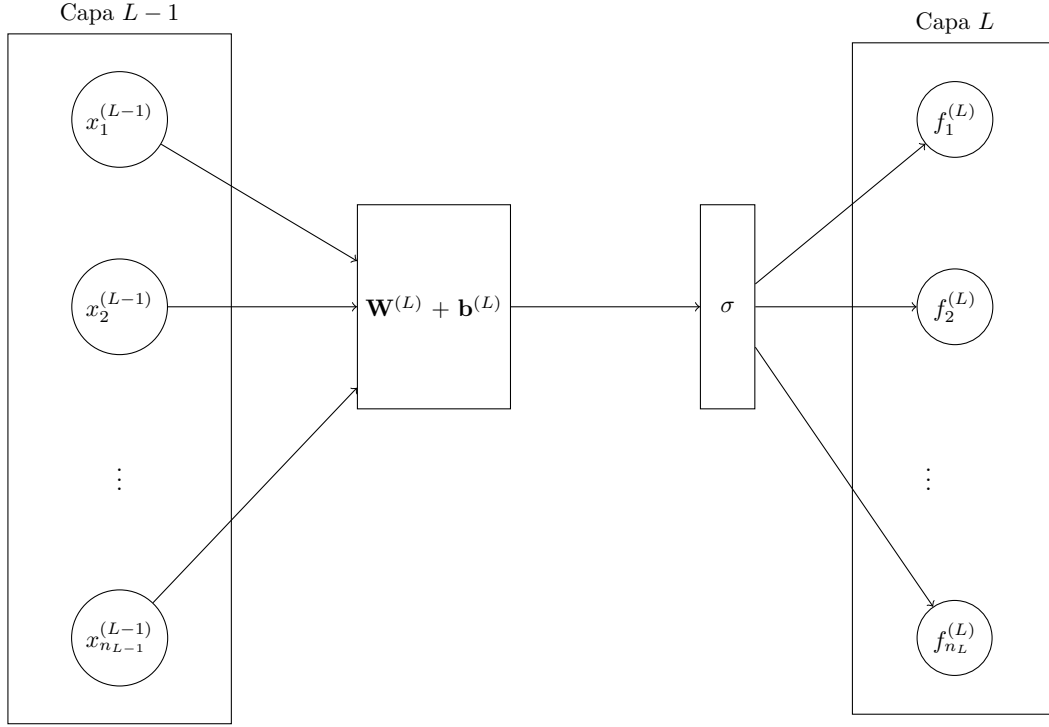


Figura 3.2: Capa de una red neuronal

Observación 1. La función de activación σ introduce no linealidad en la red neuronal, lo cual es de suma importancia para que la red pueda aprender representaciones complejas y no lineales de los datos. Algunas funciones de activación comunes son:

- **Función sigmoide:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Función ReLU (Rectified Linear Unit):** $\sigma(x) = \max(0, x)$
- **Función tanh:** $\sigma(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Con la definición anterior podemos ahora entender cómo se compone una red neuronal a partir de múltiples capas.

Definición 41 (Red neuronal). Sea $L \in \mathbb{N}$ el número de capas. Una **red neuronal** con L capas es una función

$$f(x; \theta) : \mathbb{R}^{d_{in}} \longrightarrow \mathbb{R}^{d_{out}}$$

definida como la composición

$$f(x; \theta) = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(2)} \circ f^{(1)}(x),$$

donde: $x \in \mathbb{R}^{d_{in}}$ es el vector de entrada, para cada $i \in \{1, \dots, L\}$, $f^{(i)} : \mathbb{R}^{d_{i-1}} \rightarrow \mathbb{R}^{d_i}$ es la transformación asociada a la i -ésima capa, $\theta = \{W^{(i)}, b^{(i)} \mid i = 1, \dots, L\}$ es el conjunto de parámetros de la red (matrices de pesos y vectores de sesgo), $y = f(x; \theta) \in \mathbb{R}^{d_{out}}$ es la salida de la red.

En este caso a la función $f^{(1)}$ se le conoce como la primer capa de la red, y a la capa $f^{(L)}$ se le conoce como la capa de salida. A las capas restantes se les conoce como capas ocultas

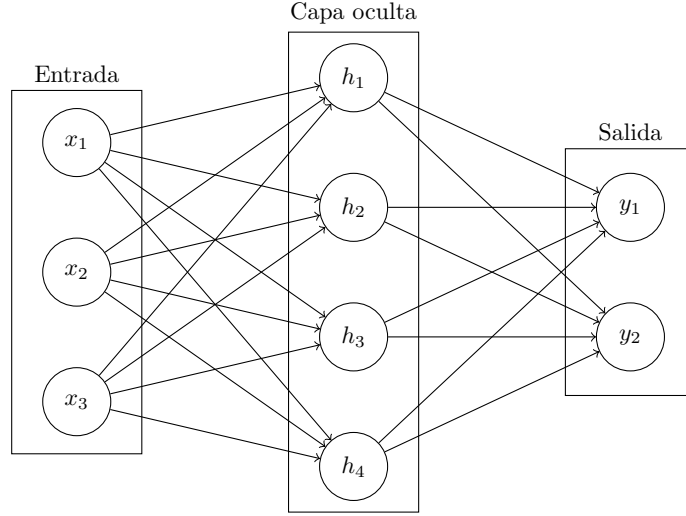


Figura 3.3: Ejemplo de red neuronal con una capa oculta

3.1.2. Entrenamiento de redes neuronales

El entrenamiento de una red neuronal implica ajustar sus parámetros θ para minimizar la función de pérdida $\mathbf{L}(y, \hat{y})$. Este proceso se lleva a cabo mediante un conjunto de datos de entrenamiento, que consiste en pares de entrada-salida (x_i, y_i) . La función de pérdida cuantifica la discrepancia entre las predicciones de la red $\hat{y}_i = f(x_i; \theta)$ y los valores reales y_i . El objetivo del entrenamiento es encontrar los parámetros θ que minimicen esta función de pérdida en promedio sobre todo el conjunto de datos. A continuación, se presentan las definiciones formales de los conceptos clave involucrados en el entrenamiento de redes neuronales.

Definición 42 (Función de pérdida). Sea $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \subset \mathcal{X} \times \mathcal{Y}$ un conjunto de datos de entrenamiento, donde $x_i \in \mathcal{X}$ son las entradas y $y_i \in \mathcal{Y}$ las etiquetas verdaderas asociadas. Sea $f(\cdot; \theta) : \mathcal{X} \rightarrow \mathcal{Y}$ una red neuronal parametrizada por $\theta \in \Theta$.

Una **función de pérdida** es un mapeo

$$L : \mathcal{Y} \times \mathcal{Y} \longrightarrow \mathbb{R}_{\geq 0},$$

tal que $L(y, \hat{y})$ mide el error incurrido al predecir $\hat{y}$ cuando el valor verdadero es y , es decir $L(y, \hat{y})$ cuantifica la discrepancia entre la predicción de la red $f(x; \theta)$ y la etiqueta verdadera y .

Como ya se ha mencionado, el proceso de entrenamiento de una red neuronal tiene como propósito ajustar los parámetros del modelo $\theta \in \Theta$ para minimizar la discrepancia entre las predicciones $f(x_i; \theta)$ y los valores reales y_i .

De manera teórica, este objetivo puede formularse a través de la **función de riesgo**, definida como la esperanza del error del modelo bajo la distribución conjunta de las variables aleatorias $(\mathbf{x}, \mathbf{y})$:

$$\mathcal{R}(\theta) = \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}} [L(y, f(x; \theta))], \quad (3.1)$$

donde $\mathcal{D}$ representa la distribución desconocida de los datos, y $L(y, f(x; \theta))$ es una función de pérdida que mide el error entre la predicción y el valor real.

Sin embargo, en la práctica esta distribución $\mathcal{D}$ es desconocida. Por ello, el riesgo teórico se aproxima mediante su **riesgo empírico**, calculado sobre una muestra finita $\{(x_i, y_i)\}_{i=1}^N$ proveniente de $\mathcal{D}$:

$$\hat{\mathcal{R}}(\theta) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i; \theta)). \quad (3.2)$$

De esta forma, el problema de aprendizaje se plantea como una tarea de *minimización del riesgo empírico* (Empirical Risk Minimization, ERM):

$$\min_{\theta \in \Theta} \hat{\mathcal{R}}(\theta) = \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i; \theta)). \quad (3.3)$$

En este contexto, la función de pérdida L cuantifica el error a nivel individual, mientras que la función de riesgo $\mathcal{R}(\theta)$ expresa dicho error en promedio —ya sea teóricamente (riesgo esperado) o empíricamente (riesgo observado).

El proceso de entrenamiento de la red busca, por tanto, minimizar una estimación del riesgo esperado mediante técnicas de optimización numérica como el *descenso del gradiente estocástico* (Stochastic Gradient Descent, SGD).

Dado que en la práctica no conocemos la verdadera distribución $p(x, y)$ no podemos calcular directamente el riesgo esperado. En su lugar definimos otro tipo de función de riesgo:

Definición 43 (Función de riesgo empírico). *Sea $\{(x_i, y_i)\}, i \in \{1, \dots, N\}$ un conjunto finito de datos de entrenamiento, la función de riesgo empírico está dada por:*

$$\hat{\mathcal{R}}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathbf{L}(y_i, f(x_i; \theta))$$

Una manera de minimizar las funciones de pérdida, es utilizando el algoritmo de Gradiente Descendente. Para ello recordemos la definición de gradiente, así como visualizaremos el algoritmo de gradiente descendente.

Definición 44 (Gradiente). *Sea $f : \Omega \subset \mathbb{R}^n \rightarrow \mathbb{R}$, una función diferenciable en x_0 . Entonces el vector cuyas componentes son las derivadas parciales de f en x_0 es el vector gradiente, denotado por ∇f , donde:*

$$\nabla f(x_0) = \left(\frac{\partial f(x_0)}{\partial x_1}, \dots, \frac{\partial f(x_0)}{\partial x_n} \right)$$

Este vector contiene la información de cuanto crece o decrece la función en un punto en específico.

Definición 45 (Descenso por gradiente). *Sea $J : \Theta \rightarrow \mathbb{R}$ una función diferenciable, denominada función objetivo, donde $\Theta \subseteq \mathbb{R}^n$ representa el espacio de parámetros. Dado un valor inicial $\theta_0 \in \Theta$ y una tasa de aprendizaje $\eta > 0$, el método de descenso por gradiente genera una sucesión $\{\theta_k\}_{k \in \mathbb{N}}$ definida recursivamente por*

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} J(\theta_k),$$

donde $\nabla_{\theta} J(\theta_k)$ denota el gradiente de J evaluado en θ_k .

Notamos que el descenso por gradiente es un método iterativo que busca minimizar una función objetivo $J(\theta)$ ajustando los parámetros θ en la dirección opuesta al gradiente de la función. La tasa de aprendizaje η controla el tamaño de los pasos que se dan en cada iteración, lo que afecta la velocidad y estabilidad de la convergencia hacia el mínimo.

Algorithm 1 Algoritmo de actualización GD

Inputs:

$J : \Theta \rightarrow \mathbb{R}$, $\mathbf{T} > 0$ número de iteraciones, η rango de aprendizaje.

Initialize:

Seleccionar $\theta^{(0)} \in \Theta$ aleatoriamente.

for $t \in \{1, \dots, \mathbf{T}\}$ **do**

Calcular $R(\theta^{(t)})$, $\nabla_{\theta^{(t)}} R(\theta^{(t)})$

if $\nabla_{\theta^{(t)}} R(\theta^{(t)}) \neq 0$ **then**

$\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta \nabla_{\theta^{(t)}} R(\theta^{(t)})$

else

Fin del algoritmo

end if

end for

A medida que va aumentando el número de capas en la red neuronal, también aumenta la complejidad de determinar los pesos de todas las conexiones de manera correcta para minimizar los errores, es por eso que nace la necesidad de implementar algoritmos más complejos para la optimización de redes multicapa, de ahí que nace el **BackPropagation**.

3.1.3. Backpropagation

El algoritmo de **backpropagation** es un método eficiente para calcular los gradientes de la función de pérdida con respecto a los parámetros de una red neuronal. Este algoritmo se basa en la regla de la cadena del cálculo diferencial y permite propagar el error desde la capa de salida hacia las capas anteriores, ajustando los pesos y sesgos de manera iterativa para minimizar la función de pérdida.

Definición 46 (Derivada función de riesgo). Sea $f(x; \theta)$ una red neuronal con parámetros θ , la derivada de su función de riesgo $R(\theta)$ se define de la siguiente forma:

$$\nabla_{\theta} R(\theta) = \nabla f^{(\mathbf{L})} \cdot \nabla f^{(\mathbf{L}-1)} \dots \nabla f^{(2)} \cdot \nabla f^{(1)}$$

A partir de la definición anterior, podemos describir el algoritmo de manera más general y dividirlo en dos partes principales: la propagación hacia adelante y la propagación hacia atrás.

Propagación hacia adelante

En esta fase, los datos de entrada se pasan a través de la red capa por capa, calculando las activaciones en cada capa utilizando los pesos y sesgos actuales. La salida final de la red se compara con la etiqueta verdadera para calcular el error utilizando una función de pérdida.

Sabemos que para cada capa (L) de una red neuronal, la salida $\mathbf{h}^{(L)}$ se calcula de manera recursiva, dado un vector de entrada $\mathbf{x}$ la red realiza las siguientes dos operaciones:

■ Preactivación

Dada la matriz de pesos $\mathbf{W}^{(L)}$, el vector de sesgos $\mathbf{b}^{(L)}$ en la capa L y $\mathbf{h}^{(L-1)}$ la salida de la capa anterior, entonces la preactivación está dada por:

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}$$

■ Activación

Dada σ una función de activación (sigmoide, ReLu, etc), y $\mathbf{z}^{(L)}$ la preactivación de la capa L , entonces la activación en la capa L está dada por:

$$\mathbf{h}^{(L)} = \sigma(\mathbf{z}^{(L)})$$

De esta forma la salida de la última capa se compara con la etiqueta verdadera $\mathbf{y}$ utilizando alguna función de pérdida $\mathbf{J}(\mathbf{h}^{(L)}, \mathbf{y})$ para calcular el error de la red.

Propagación hacia atrás

Para minimizar el error en la función de pérdida utilizamos la definición 45, por una serie de pasos dados de la siguiente manera:

■ Error en la capa de salida

Sea $\frac{\partial \mathbf{J}}{\partial \mathbf{h}^{(L)}}$ el gradiente de la función de pérdida con respecto a la salida de la red, $\sigma'(\mathbf{z}^{(L)})$, la derivada de la función de activación de la última capa, entonces el error en la capa de salida está dado por:

$$\delta^{(L)} = \frac{\partial \mathbf{J}}{\partial \mathbf{z}^{(L)}} = \frac{\partial \mathbf{J}}{\partial \mathbf{h}^{(L)}} \cdot \sigma'(\mathbf{z}^{(L)})$$

■ Propagación del error hacia atrás

Sea $\delta^{(L+1)}$ el error de la capa siguiente y $\sigma'(\mathbf{z}^{(L)})$ la derivada de la función de activación de la capa L , entonces la propagación del error se da mediante la siguiente expresión:

$$\delta^{(L)} = \left(\mathbf{W}^{(L+1)} \right)^T \delta^{(L+1)} \cdot \sigma'(\mathbf{z}^{(L)})$$

■ Gradiente de los parámetros

Para calcular los gradientes de los parametros, vemos que se realiza de la siguiente forma:

- Sea $\mathbf{h}^{(L-1)}$ la salida de la capa anterior, entonces el gradiente de los pesos está dado por:

$$\frac{\partial \mathbf{J}}{\partial \mathbf{W}^{(L)}} = \delta^{(L)} \cdot \left(\mathbf{h}^{(L-1)} \right)^T$$

- Y para los sesgos se calcula de la siguiente forma:

$$\frac{\partial \mathbf{J}}{\partial \mathbf{b}^{(L)}} = \delta^{(L)}$$

Actualización de los parámetros

Una vez que se tienen los pesos y sesgos calculados de la red, se actualizan utilizando descenso del gradiente, de esta forma la actualización está dada por:

- **Pesos:**

$$W^{(L)} = W^{(L)} - \eta \frac{\partial \mathbf{J}}{\partial W^{(L)}}$$

- **Bias:**

$$b^{(L)} = b^{(L)} - \eta \frac{\partial \mathbf{J}}{\partial b^{(L)}}$$

Con lo observado anteriormente, podemos definir formalmente el algoritmo **backpropagation**.

Algorithm 2 Algoritmo Backpropagation

```

1: Initialize Inicializamos pesos  $W^{[l]}$  y bias  $b^{[l]}$  de manera aleatoria para cada capa  $l = 1, \dots, L$ 
2: for cada época do
3:   for cada ejemplo de entrenamiento  $(x, y)$  do
4:     Propagación hacia adelante:
5:     Asignamos  $a^{[0]} = x$ 
6:     for each layer  $l = 1, \dots, L$  do
7:        $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$ 
8:        $a^{[l]} = g(z^{[l]})$ 
9:     end for
10:    Calculamos la pérdida  $J(a^{[L]}, y)$ 
11:    Backward Pass:
12:    Calculamos el error para cada capa de salida:
13:     $\delta^{[L]} = \frac{\partial J}{\partial a^{[L]}} \cdot g'(z^{[L]})$ 
14:    for cada capa  $l = L - 1, \dots, 1$  do
15:      Propagación del error hacia atrás:
16:       $\delta^{[l]} = (W^{[l+1]})^\top \delta^{[l+1]} \cdot g'(z^{[l]})$ 
17:      Calculamos los gradientes:
18:       $\frac{\partial J}{\partial W^{[l]}} = \delta^{[l]} \cdot (a^{[l-1]})^\top$ 
19:       $\frac{\partial J}{\partial b^{[l]}} = \delta^{[l]}$ 
20:    end for
21:    Actualización de parámetros (Descenso del gradiente):
22:    for each layer  $l = 1, \dots, L$  do
23:       $W^{[l]} = W^{[l]} - \eta \cdot \frac{\partial J}{\partial W^{[l]}}$ 
24:       $b^{[l]} = b^{[l]} - \eta \cdot \frac{\partial J}{\partial b^{[l]}}$ 
25:    end for
26:  end for
27: end for

```

De esta forma hemos definido los componentes principales de una red neuronal, así como los algoritmos necesarios para su entrenamiento. A continuación, nos enfocaremos en un tipo específico de redes neuronales conocidas como **redes convolucionales (CNN)**, que son especialmente efectivas para procesar datos con estructura espacial, como imágenes y series temporales.

3.2. Redes convolucionales CNN

Las **redes neuronales convolucionales (CNN)** son una clase especializada de redes neuronales diseñadas para procesar datos que poseen una estructura de malla o grilla, como imágenes y series de tiempo. Estas redes se emplean comúnmente en tareas de procesamiento

de imágenes, donde los datos se organizan en una malla bidimensional, o en el caso de las series temporales, que pueden representarse como una malla unidimensional con muestras tomadas a lo largo de intervalos de tiempo.

A diferencia de las redes neuronales tradicionales, donde se utiliza la multiplicación general de matrices, las CNN aplican la operación de convolución en al menos una de sus capas. Esto les permite extraer automáticamente características locales de los datos, como bordes, texturas o patrones, lo que mejora significativamente su capacidad para procesar datos con una estructura espacial clara.

Estudiar las CNN es fundamental como antecedente para entender las redes gráficas convolucionales (GCN), ya que comparten la idea central de la convolución, pero aplicadas a estructuras de datos más complejas, como las gráficas. En esta sección, exploraremos la importancia de las CNN como base teórica, profundizando en su arquitectura y en las matemáticas que subyacen a este tipo particular de redes. La mayoría de las definiciones fueron obtenidas del libro [?].

3.2.1. Operación de convolución

Para entender las redes convolucionales, es esencial comprender primero la operación matemática de la convolución. A continuación, se presentan las definiciones clave relacionadas con la convolución en diferentes contextos.

Definición 47 (Convolución). Sean $x, w : \mathbb{R} \rightarrow \mathbb{R}$ dos funciones integrables. La **convolución** de x y w , denotada por $x * w$, es la función

$$(x * w) : \mathbb{R} \rightarrow \mathbb{R}$$

definida, para todo $t \in \mathbb{R}$, por

$$(x * w)(t) = \int_{-\infty}^{\infty} x(a) w(t - a) da,$$

siempre que dicha integral exista.

En el contexto de las redes neuronales convolucionales, el primer término (la función x) se refiere a la entrada, y el segundo término (la función w) es el kernel o filtro.

Dado que en muchos escenarios, el tipo de datos es discreto, se maneja otro tipo de convolución, llamada convolución discreta.

Definición 48 (Convolución discreta). Sean $x, w : \mathbb{Z} \rightarrow \mathbb{R}$ dos sucesiones absolutamente sumables. La **convolución discreta** de x y w , denotada por $x * w$, es la sucesión

$$(x * w) : \mathbb{Z} \rightarrow \mathbb{R}$$

definida, para todo $n \in \mathbb{Z}$, por

$$(x * w)(n) = \sum_{a=-\infty}^{\infty} x(a) w(n - a),$$

siempre que dicha serie sea convergente.

Finalmente, para el uso particular del procesamiento de imágenes en dos dimensiones, podemos utilizar la convolución discreta en dos dimensiones.

Definición 49 (Convolución discreta en dos dimensiones). Sean $\mathbf{I}, \mathbf{K} : \mathbb{Z}^2 \rightarrow \mathbb{R}$ dos funciones (o matrices discretas) absolutamente sumables. La **convolución discreta bidimensional** de $\mathbf{I}$ y $\mathbf{K}$, denotada por $\mathbf{I} * \mathbf{K}$, es la función

$$(\mathbf{I} * \mathbf{K}) : \mathbb{Z}^2 \rightarrow \mathbb{R}$$

definida, para todo $(i, j) \in \mathbb{Z}^2$, por

$$(\mathbf{I} * \mathbf{K})(i, j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \mathbf{I}(m, n) \mathbf{K}(i - m, j - n),$$

siempre que la doble suma sea convergente.

Observación 2. La operación de convolución es conmutativa, es decir,

$$\mathbf{I} * \mathbf{K} = \mathbf{K} * \mathbf{I}.$$

Demostración. Sean $(i, j) \in \mathbb{Z}^2$. Por definición de convolución discreta en dos dimensiones, tenemos:

$$(\mathbf{I} * \mathbf{K})(i, j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \mathbf{I}(m, n) \mathbf{K}(i - m, j - n).$$

Observamos que cada suma converge, por lo que podemos intercambiar el orden de las sumas y reordenar los términos. Haciendo el cambio de variables $u = i - m$ y $v = j - n$, obtenemos:

$$\begin{aligned} (\mathbf{I} * \mathbf{K})(i, j) &= \sum_{u=-\infty}^{\infty} \sum_{v=-\infty}^{\infty} \mathbf{I}(i - u, j - v) \mathbf{K}(u, v) \\ &= \sum_{u=-\infty}^{\infty} \sum_{v=-\infty}^{\infty} \mathbf{K}(u, v) \mathbf{I}(i - u, j - v) \\ &= (\mathbf{K} * \mathbf{I})(i, j) \end{aligned}$$

Además, observamos que la segunda igualdad es posible, debido a la conmutatividad del producto en cada término de la suma. Cómo esto es válido para todo $(i, j) \in \mathbb{Z}^2$, concluimos que:

$$\mathbf{I} * \mathbf{K} = \mathbf{K} * \mathbf{I}.$$

□

En consecuencia, la expresión anterior puede reescribirse de manera equivalente como

$$(\mathbf{K} * \mathbf{I})(i, j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \mathbf{K}(m, n) \mathbf{I}(i - m, j - n).$$

3.2.2. Propiedades de convolución

Una de las propiedades más importantes de la operación de convolución es su comportamiento bajo ciertas transformaciones, específicamente la invarianza y la equivarianza. Estas propiedades son fundamentales para entender cómo las redes convolucionales pueden aprender características relevantes de los datos de entrada. A continuación, se presentan las definiciones formales de invarianza y equivarianza.

Definición 50 (Invarianza). Sea $f : \mathbf{X} \rightarrow \mathbf{Y}$ una función, $\mathbf{T}$ una transformación en $\mathbf{X}$. Se dice que una función f es invariante bajo $\mathbf{T}$ si:

$$f(\mathbf{T}(x)) = f(x), \quad \forall x \in \mathbf{X}$$

Definición 51 (Equivarianza). Sean f, g dos funciones tales que $f : \mathbf{X} \rightarrow \mathbf{Y}$, $g : \mathbf{Y} \rightarrow \mathbf{X}$. Se dice que f es equivariante bajo g si:

$$f(g(x)) = g(f(x)) \quad \forall x \in \mathbf{X}$$

De lo anterior podemos observar que la invarianza implica que la salida de la función no cambia cuando se aplica una transformación a la entrada, mientras que la equivarianza implica que la transformación de la entrada se refleja de manera consistente en la salida. A continuación, demostraremos que la operación de convolución es equivariante bajo traslaciones.

Teorema 6 (Invarianza de la convolución bajo traslaciones). Sea $f, k \in L^1(\mathbb{R})$ y sea $\mathbf{T}_a$ el operador de traslación definido por

$$(\mathbf{T}_a f)(x) = f(x - a), \quad a \in \mathbb{R}.$$

Entonces, para todo $x \in \mathbb{R}$, se cumple que

$$(\mathbf{T}_a f * k)(x) = (f * k)(x - a).$$

Demostración. Por definición de la convolución en $\mathbb{R}$, tenemos que

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} (\mathbf{T}_a f)(t) k(x - t) dt.$$

Sustituyendo la definición de la traslación, se obtiene

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(t - a) k(x - t) dt.$$

Haciendo el cambio de variable $u = t - a$, de modo que $du = dt$ y $t = u + a$, se sigue que

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(u) k(x - (u + a)) du.$$

Reordenando los términos en el argumento de k , obtenemos

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(u) k((x - a) - u) du.$$

Por la definición de la convolución, esto equivale a

$$(f * k)(x - a) = \int_{\mathbb{R}} f(u) k((x - a) - u) du.$$

Por lo tanto,

$$(\mathbf{T}_a f * k)(x) = (f * k)(x - a), \quad \forall x \in \mathbb{R}.$$

□

De esta forma, demostramos que la operación convolución para funciones definidas en $\mathbb{R}$ son equivariantes con respecto a las traslaciones. Es decir, que en el contexto de las redes neuronales, si trasladamos la entrada de la red, significa que la salida estará trasladada en la misma medida, lo cual es útil cuando se aplica a múltiples ubicaciones en la entrada.

3.2.3. Filtros y mapas de características

En el contexto de las redes neuronales convolucionales, los **filtros** (o kernels) son matrices de pesos que se utilizan para extraer características locales de los datos de entrada mediante la operación de convolución. A continuación, se presentan las definiciones clave relacionadas con los filtros y los mapas de características.

Definición 52 (Kernel (Filtro)). Sea $\mathbf{X} \in \mathbb{R}^{m \times n}$ una matriz que representa los datos de entrada. Un **kernel** (o **filtro**) es una matriz de pesos

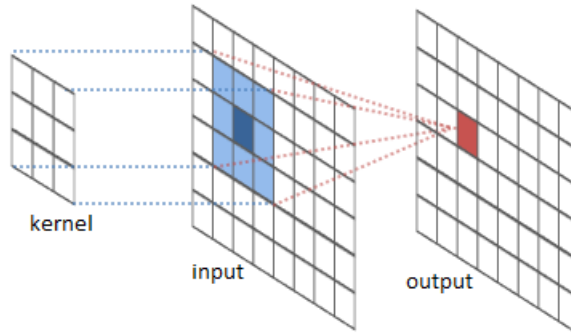
$$\mathbf{K} \in \mathbb{R}^{h \times w}, \quad \text{con } 1 \leq h \leq m, 1 \leq w \leq n.$$

tal que la acción del kernel sobre la entrada $\mathbf{X}$ se define mediante la **convolución discreta bidimensional**, dada por

$$(\mathbf{K} * \mathbf{X})(i, j) = \sum_{u=1}^h \sum_{v=1}^w \mathbf{X}(i + u - 1, j + v - 1) \mathbf{K}(u, v),$$

para todo (i, j) tal que los índices sean válidos en la matriz resultante.

El desplazamiento en los índices por -1 se introduce para asegurar la alineación del kernel respecto al elemento central de la región de entrada sobre la cual actúa.



A continuación, enunciaremos algunas propiedades clave de los filtros.

Definición 53 (Propiedades del kernel). Sea $\mathbf{K} \in \mathbb{R}^{h \times w}$ un filtro (kernel) discreto de dimensiones $h \times w$. Entonces, se definen las siguientes propiedades:

- **Dimensión.** La dimensión del kernel viene dada por el producto de sus componentes espaciales:

$$\dim(\mathbf{K}) = h \times w.$$

- **Simetría.** El kernel $\mathbf{K}$ se dice simétrico si sus valores son invariantes bajo reflexión respecto al centro del filtro, es decir:

$$\mathbf{K}(i, j) = \mathbf{K}(h - i + 1, w - j + 1), \quad \forall i \in \{1, \dots, h\}, j \in \{1, \dots, w\}.$$

- **Normalización.** El kernel $\mathbf{K}$ está normalizado si la suma total de sus coeficientes es igual a la unidad:

$$\sum_{i=1}^h \sum_{j=1}^w \mathbf{K}(i, j) = 1.$$

Con las definiciones anteriores, podemos definir el concepto de mapa de características, que es el resultado de aplicar un filtro a una entrada mediante la operación de convolución.

Definición 54 (Mapa de características). Sea $\mathbf{I} \in \mathbb{R}^{m \times n}$ un tensor (o matriz) de entrada y sea $\mathbf{K} \in \mathbb{R}^{h \times w}$ un kernel (o filtro) de convolución. El **mapa de características** asociado a $\mathbf{I}$ y $\mathbf{K}$ se define como la matriz $\mathbf{F} \in \mathbb{R}^{(m-h+1) \times (n-w+1)}$ cuyos elementos están dados por:

$$\mathbf{F}(i, j) = (\mathbf{I} * \mathbf{K})(i, j) = \sum_{u=1}^h \sum_{v=1}^w \mathbf{I}(i + u - 1, j + v - 1) \mathbf{K}(u, v),$$

para todo $1 \leq i \leq m - h + 1$ y $1 \leq j \leq n - w + 1$.

Adicionalmente, existen dos conceptos importantes en el contexto de las redes convolucionales: el **stride** y el **padding**. El **stride** se refiere al paso con el que se mueve el filtro sobre la entrada durante la operación de convolución, mientras que el **padding** implica agregar bordes adicionales a la entrada para controlar las dimensiones del mapa de características resultante.

Definición 55 (Stride). Sea $\mathbf{I} \in \mathbb{R}^{m \times n}$ una matriz de entrada y $\mathbf{K} \in \mathbb{R}^{h \times w}$ un kernel de convolución. Definimos el **stride** (o paso de desplazamiento) como un entero positivo $s \in \mathbb{N}$ que determina el número de posiciones que se desplaza el kernel $\mathbf{K}$ sobre la entrada $\mathbf{I}$ en cada dirección.

De este modo, los índices (i, j) del mapa de características $\mathbf{F}$ corresponden a las posiciones:

$$(i, j) = (p \cdot s, q \cdot s), \quad \text{para } p, q \in \mathbb{N}.$$

Por tanto, los elementos del mapa de características se definen como:

$$\mathbf{F}(p, q) = (\mathbf{I} *_s \mathbf{K})(p, q) = \sum_{u=1}^h \sum_{v=1}^w \mathbf{I}(p \cdot s + u - 1, q \cdot s + v - 1) \mathbf{K}(u, v),$$

donde $*_s$ denota la convolución con stride s .

Definición 56 (Padding). Sea $\mathbf{I} \in \mathbb{R}^{m \times n}$ una entrada y sea $\rho \in \mathbb{N}$ el tamaño del **padding**. Definimos la función de padding

$$\mathbf{p}_\rho : \mathbb{R}^{m \times n} \longrightarrow \mathbb{R}^{(m+2\rho) \times (n+2\rho)}$$

como

$$\mathbf{p}_\rho(\mathbf{I})(i, j) = \begin{cases} \alpha, & \text{si } i \leq \rho \text{ o } i > m + \rho \text{ o } j \leq \rho \text{ o } j > n + \rho, \\ \mathbf{I}(i - \rho, j - \rho), & \text{en otro caso.} \end{cases}$$

donde $\alpha \in \mathbb{R}$ es el valor con el que se rellenan los bordes (por ejemplo, $\alpha = 0$ en el caso de zero-padding).

Otra operación común en las redes convolucionales es el **pooling**, que se utiliza para reducir las dimensiones espaciales de los mapas de características, ayudando a disminuir la cantidad de parámetros y computación en la red, además de controlar el sobreajuste.

Definición 57 (Max Pooling). Sea $\mathbf{X} \in \mathbb{R}^{m \times n}$ una matriz de entrada, y sean $h, w \in \mathbb{N}$ las dimensiones de la ventana pooling, y $s \in \mathbb{N}$ el parámetro de stride. Definimos la operación de **max pooling** como una función

$$\mathbf{M} : \mathbb{R}^{m \times n} \longrightarrow \mathbb{R}^{p \times q},$$

dada por

$$\mathbf{M}(\mathbf{X})(i, j) = \max_{0 \leq u < h, 0 \leq v < w} \mathbf{X}(i \cdot s + u, j \cdot s + v),$$

para todo $i = 0, \dots, p-1$ y $j = 0, \dots, q-1$, donde $p = \lfloor \frac{m-h}{s} \rfloor + 1$ y $q = \lfloor \frac{n-w}{s} \rfloor + 1$.

Definición 58 (Average Pooling). Sea $\mathbf{X} \in \mathbb{R}^{m \times n}$ una matriz de entrada, y sean $h, w \in \mathbb{N}$ las dimensiones de la ventana de pooling, y $s \in \mathbb{N}$ el parámetro de stride. Definimos la operación de **average pooling** como una función

$$\mathbf{A} : \mathbb{R}^{m \times n} \longrightarrow \mathbb{R}^{p \times q},$$

dada por

$$\mathbf{A}(\mathbf{X})(i, j) = \frac{1}{h \cdot w} \sum_{u=0}^{h-1} \sum_{v=0}^{w-1} \mathbf{X}(i \cdot s + u, j \cdot s + v),$$

para todo $i = 0, \dots, p-1$ y $j = 0, \dots, q-1$, donde $p = \lfloor \frac{m-h}{s} \rfloor + 1$ y $q = \lfloor \frac{n-w}{s} \rfloor + 1$.

Con lo anterior hemos definido los conceptos fundamentales relacionados con las redes neuronales convolucionales, incluyendo la operación de convolución, los filtros, los mapas de características, el stride, el padding y las operaciones de pooling. A continuación, exploraremos la arquitectura típica de una CNN y cómo estos componentes se integran para formar una red completa.

3.2.4. Arquitectura de una CNN

A continuación, presentamos la definición formal de la arquitectura de una red neuronal convolucional (CNN).

Definición 59 (Arquitectura de una red neuronal convolucional (CNN)). Sea $f : \mathbb{R}^{m \times n \times d} \rightarrow \mathbb{R}^k$ una función que representa el modelo completo de una red neuronal convolucional. La arquitectura de una CNN se define como la composición de tres funciones principales:

$$\mathbf{y} = (\mathbf{g} \circ \mathbf{P} \circ \mathbf{C})(\mathbf{X}),$$

donde $\mathbf{C}$ denota la capa convolucional, $\mathbf{P}$ la capa de pooling y $\mathbf{g}$ la capa completamente conectada, definidas como sigue.

La **capa convolucional** se define por

$$\mathbf{C}(\mathbf{X}) = (\mathbf{K} * \mathbf{X}) + \mathbf{b},$$

donde $\mathbf{K}$ representa el conjunto de filtros convolucionales y $\mathbf{b}$ el sesgo asociado.

La **capa de pooling**, en este caso correspondiente al max pooling, está dada por

$$\mathbf{P}(\mathbf{C}(\mathbf{X}))(i, j, c) = \max_{0 \leq u < h, 0 \leq v < w} \mathbf{C}(\mathbf{X})(i \cdot s + u, j \cdot s + v, c),$$

donde (h, w) son las dimensiones de la ventana y s el desplazamiento o stride.

Finalmente, la **capa completamente conectada** se expresa como

$$\mathbf{y} = \mathbf{g}(\mathbf{z}) = \sigma(\mathbf{W}\mathbf{z} + \mathbf{b}),$$

donde $\mathbf{z}$ es la versión aplanada de $\mathbf{P}(\mathbf{C}(\mathbf{X}))$, $\mathbf{W}$ la matriz de pesos y σ una función de activación no lineal.

Con lo definido anteriormente, se ha establecido una base sólida para comprender las redes neuronales convolucionales (CNN). A continuación, nos adentraremos en el estudio de las redes neuronales gráficas (GNN), que extienden los conceptos de las CNN para trabajar con datos estructurados en forma de gráficas.

3.3. Redes Neuronales Gráficas

Las **Redes Neuronales Gráficas** (GNN, por sus siglas en inglés) son una clase de modelos de aprendizaje automático diseñados para trabajar con datos estructurados en forma de gráficas. A diferencia de las redes neuronales tradicionales que operan sobre datos tabulares o secuenciales, las GNN están diseñadas para capturar las relaciones y dependencias entre los nodos y las aristas de una gráfica.

Cómo paso inicial, estableremos algunas definiciones, que nos ayudarán a entender y formalizar el concepto de GNN. Puesto que hemos definido y estudiado las definiciones básicas de gráficas en la sección 2, no nos detendremos en ellas. Las definiciones que se presentan a continuación están basadas en el libro [?]

3.3.1. Encoders y Decoders

En el contexto de las Redes Neuronales Gráficas (GNN), los conceptos de **encoder** y **decoder** son fundamentales para comprender cómo se procesan y representan los datos

estructurados en forma de gráficas. A continuación, se presentan las definiciones formales de estos conceptos.

Definición 60 (Encoder). *Sea $G = (V, E)$ una gráfica, donde V denota el conjunto de vértices y E el conjunto de aristas. Un encoder es una función*

$$\phi : V \longrightarrow \mathbb{R}^d$$

que asigna a cada vértice $v \in V$ un encaje $z_v \in \mathbb{R}^d$, el cual se obtiene a partir de la información local de v , sus vecinos $\mathcal{N}(v)$ y las aristas de la gráfica $E(G)$, esto es:

$$\phi(v, \mathcal{N}(v), E(G)) = z_v, \quad \text{con } z_v \in \mathbb{R}^d.$$

Por simplicidad denotaremos $\phi(v, \mathcal{N}(v), E(G)) := \phi(v)$.

De forma similar a la matriz de adyacencia de una gráfica, a partir de la definición de encoder, podemos definir la matriz de encajes de una gráfica.

Definición 61 (Matriz de encajes). *Sea $G = (V, E)$ una gráfica, con $|V| = \nu_G$. La matriz de encajes es una matriz*

$$\mathbf{Z} \in \mathbb{R}^{\nu_G \times d}$$

cuyas filas corresponden a los encajes de los vértices. Para cada $v \in V$, el vector de encaje asociado se denota por

$$\phi(v) = \mathbf{Z}[v],$$

donde $\mathbf{Z}[v]$ representa la fila de $\mathbf{Z}$ correspondiente al vértice v , y $\phi(v) = z_v \in \mathbb{R}^d$ es el vector de encaje del nodo v .

A su vez, podemos definir el concepto de decoder como la función que toma los encajes producidos por el encoder y genera una predicción en el espacio de salida. En el contexto de autoencoders sobre grafos, el decoder busca reconstruir la estructura original de la gráfica (por ejemplo, su matriz de adyacencia) o inferir propiedades derivadas de ella, utilizando la información contenida en los vectores de encaje.

Definición 62 (Decoder). *Sea $z_v \in \mathbb{R}^d$ el vector de encaje asociado a un vértice $v \in V$, obtenido mediante un encoder. Se denomina decoder a una función*

$$\psi : \mathbb{R}^d \longrightarrow \mathcal{Y},$$

que asigna a cada representación latente z_v una predicción en el espacio de salida $\mathcal{Y}$, de modo que

$$\psi(z_v) = \hat{y}.$$

Los conceptos de encoder y decoder son de suma importancia en las GNN, ya que permiten transformar la información estructural de las gráficas en representaciones vectoriales útiles para tareas de aprendizaje automático, como clasificación de nodos, predicción de enlaces y generación de gráficas. Más adelante, se explicarán como se usan estos conceptos en las arquitecturas de redes que se utilizaron para los objetivos de este trabajo.

3.3.2. Propagación de Mensajes Neuronales

El concepto de **Propagación de Mensajes Neuronales** se refiere al mecanismo mediante el cual las representaciones de los vértices de una gráfica, se actualizan mediante el intercambio de información de su vecindario. En cada paso del mecanismo, los nodos intercambian mensajes que contienen información sobre el estado en que se encuentran para luego combinarse y actualizar su información interna.

Esto viene motivado por la necesidad de capturar relaciones y dependencias entre los vértices de una gráfica para estudiar su comportamiento estructural.

El concepto de **Propagación de Mensajes Neuronales**, permite estudiar a la gráfica, no sólo a nivel de característica de vértice, sino también a nivel estructural de gráfica.

Descripción general del mecanismo

Cómo se mencionó en la motivación, durante cada iteración del mecanismo en una GNN (Red Neuronal Gráfica), los encajes ocultos $h_u^{(k)}$ correspondientes a cada nodo $u \in V$, son actualizadas de acuerdo a la información recopilada del vecinadrio de u .

El mecanismo puede ser definido de la siguiente forma:

Definición 63 (Propagación de mensajes neuronales). *Sea $G = (V, E)$ una gráfica, $N_G(v)$ el vecindario del nodo $v \in V$, $h_u^{(t)}$ los encajes ocultos de cada nodo u , y $\mathcal{A}$ una función de agregación que combina los encajes de los vecinos de v . Entonces, el mecanismo de propagación de mensajes se define como:*

$$m_v^{(t)} = \mathcal{A}(\{h_u^{(t)} : u \in N_G(v)\}).$$

Una vez que el vértice a recibido el mensaje, la actualización de su información es el resultado de la combinación del mensaje recibido, con la información actual que tenía y se define de la siguiente forma:

Definición 64 (Actualización de estados neuronales). *Sea $h_v^{(t-1)}$ el encaje oculto del nodo v en la capa anterior $(t-1)$, y $m_v^{(t)}$ el mensaje agregado en la capa t . Entonces, la actualización del estado del nodo está dada por:*

$$h_v^{(t)} = \mathcal{U}(h_v^{(t-1)}, m_v^{(t)}),$$

donde $\mathcal{U}$ es una función de actualización, típicamente implementada mediante una red neuronal, por ejemplo, una capa lineal seguida de una función de activación (como ReLU).

Después de K iteraciones del mecanismo, podemos utilizar la salida del final de la capa para definir los encajes de cada vértice, es decir:

$$z_u = h_u^{(K)}.$$

3.3.3. La GNN fundamental

Hasta el momento, se ha estudiado el marco de las GNN de una manera un poco abstracta utilizando las funciones que definimos en la sección anterior e iterando K veces.

Para introducir una manera de abordarlas de forma más general, introducimos el modelo GNN más básico. El modelo GNN fundamental es definido por:

$$h_u = \phi \left(x_u, \bigoplus_{v \in \mathcal{N}(u)} \psi(x_u, x_v) \right)$$

Donde:

- h_u es la nueva representación del nodo u después de aplicar una capa de la GNN
- x_u Representa la característica del nodo u antes de la actualización.
- $\psi(x_u, x_v)$ Es una función de transformación que toma las características del nodo u y de su vecino v , produciendo una especie de mensaje o interacción entre ambos nodos.
- $\bigoplus_{v \in \mathcal{N}(u)}$ Es una operación de agregación como puede ser suma, media, max pooling, etc, sobre los vecinos de u es decir sobre las transformaciones producidas por la función $\psi(\cdot)$.
- $\phi(\cdot)$ es la función de actualización que toma las características originales del nodo $u(x_u)$ y el resultado de la agregación de los mensajes de sus vecinos para producir una nueva representación del nodo u .

En términos generales, el comportamiento lo podemos describir a continuación:

1. Mensajes entre nodos vecinos ($\psi(x_u, x_v)$)

La función ψ calcula la interacción entre las características del nodo u y las características de cada uno de sus vecinos v . Esto permite capturar la relación entre los nodos de forma que dependa de sus características.

2. Agregación ($\bigoplus$)

Después de calcular las interacciones entre u y sus vecinos, estas interacciones se agregan mediante una operación $\bigoplus$, que comúnmente (cómo se vió anteriormente) es una suma, media o máximo. La agregación tiene como objetivo combinar toda la información recibida entre los vecinos de u en un solo término.

3. Actualización(ϕ)

Finalmente, la función ϕ toma la característica original del nodo u, x_u y el resultado de la agregación para producir la nueva representación h_u . La función ϕ puede ser una red neuronal feedforward o cualquier otra transformación no lineal que aprenda a integrar la información de u con la de sus vecinos.

En resumen, el modelo anterior expresa el proceso de **message passing** de una GNN, donde los mensajes de los vecinos de un nodo se combinan para actualizar la representación del nodo central, aprovechando las simetrías presentes en las gráficas.

3.3.4. Redes Gráficas Convolucionales (GCNs)

Definición 65 (Arquitectura de una GNN). *Sea G una gráfica, A su matriz de adyacencia F una función de activación, Φ los parámetros de la red, entonces la arquitectura de una GCN está definida por:*

$$\begin{aligned}\mathbf{H}_1 &= \mathbf{F}[\mathbf{X}, \mathbf{A}, \Phi_0] \\ \mathbf{H}_2 &= \mathbf{F}[\mathbf{H}_1, \mathbf{A}, \Phi_1] \\ &\vdots = \vdots \\ \mathbf{H}_K &= \mathbf{F}[\mathbf{H}_{K-1}, \mathbf{A}, \Phi_{K-1}]\end{aligned}$$

Donde $\mathbf{X}$ es la entrada y $\mathbf{H}_K$ contiene los encajes de nodos modificados.

Definición 66 (Equivarianza). *Sea F una función de activación H la matriz de encajes ocultos de nodos, P una matriz de permutación, decimos que una capa es equivariante si:*

$$\mathbf{H}_{K+1}\mathbf{P} = \mathbf{F}[\mathbf{H}_K\mathbf{P}, \mathbf{P}^T\mathbf{A}\mathbf{P}, \Phi_K]$$

Donde A es la matriz de adyacencia para la gráfica G

Pointnet utiliza equivarianza y lo podemos mostrar. No importa la gráfica

Capítulo 4

Construcción de algoritmo de muestreo

En los últimos años, los avances en el campo del *Deep Learning* y la Inteligencia Artificial han transformado distintas disciplinas como la visión por computadora, el análisis de datos y el reconocimiento de patrones. Estas herramientas se han construido con bases matemáticas sólidas, como el Análisis matemático, el álgebra lineal y la estadística, las cuales han sido ampliamente documentadas y desarrolladas en textos con el que se ha citado en repetidas ocasiones en esta Tesis *Deep Learning* de Goodfellow et al (2016). Sin embargo la relación entre las matemáticas y el aprendizaje profundo no se limita únicamente al uso de conceptos matemáticos para el desarrollo de modelos; por el contrario, podemos utilizar dichos modelos para explorar y analizar estructuras de objetos matemáticos complejos (Bronstein et al, 2021). Este capítulo aborda el estudio de nudos de Lissajous, un tipo especial de nudo generado por funciones armónicas tridimensionales que se modelan mediante parámetros periódicos, los cuales presentan una serie de configuraciones topológicas que pueden ser modeladas y analizadas a través de herramientas modernas de aprendizaje automático y teoría de gráficas. En este contexto, el objetivo principal es clasificar con base en una serie de propiedades de estos nudos, de las cuales dentro de todas estas se encuentra la característica de Euler, una propiedad topológica fundamental que describe la conectividad y la estructura de estos objetos. Para ello, emplearemos arquitecturas avanzadas de aprendizaje profundo, como las Redes Neuronales Gráficas que hemos descrito en capítulos anteriores, que han demostrado ser herramientas efectivas para analizar estructuras no euclidianas (Xu et al 2019). A lo largo de este capítulo, se detallará la metodología empleada, que combina técnicas de visualización de gráficas, empleo de algoritmos de clasificación utilizando GNNs, así como una serie de algoritmos de muestreo que se utilizan para obtener mejores resultados. Este enfoque resalta cómo las herramientas existentes de Deep Learning pueden extenderse más allá de los problemas tradicionales de clasificación y predicción, ofreciendo nuevas perspectivas de resolver problemas fundamentales en matemáticas y teoría de nudos.

4.0.1. Superficies estudiadas

La metodología seguida para estudiar nudos de Lissajous desde un enfoque basado en gráficas comienza con la generación de superficies en $\mathbb{R}^3$ a partir de las curvas de Lissajous

”engordadas” para convertirlas en objetos con volumen. Estos objetos se triangulan utilizando la librería **trimesh** del lenguaje de programación Python, una herramienta robusta para manipulación y análisis de geometrías tridimensionales. Este paso permite representar la superficie del nudo mediante una malla compuesta por vértices y caras triangulares. Posteriormente esta malla se convierte en una gráfica mediante otra librería de Python llamada **networkX**, donde cada nodo representa un vértice de la superficie triangulada y cada arista define la conectividad entre vértices adyacentes en la superficie. Esta representación gráfica es fundamental para analizar las propiedades estructurales de los nudos ya que facilita su visualización y procesamiento mediante técnicas de aprendizaje profundo basadas en gráficas, como las GNNs. A lo largo del proceso, a parte de utilizar la librería **networkX** para la construcción, se utiliza **matplotlib** para su visualización, lo que permite inspeccionar visualmente los resultados y evaluar las características topológicas de los nudos modelados.

4.0.2. Métodos de Muestreo

Durante el análisis de las estructuras de la gráfica, se identificó un problema relacionado con el procesamiento debido a la naturaleza del entorno local. Para abordar este desafío, se diseñaron una serie de métodos y técnicas que permitieron realizar un muestreo efectivo sobre las superficies y nodos del grafo, sin comprometer la preservación de su estructura original.

4.0.3. Orden por árbol generador mínimo

Uno de los desafíos al obtener muestras de la estructura de un grafo es definir un método de recorrido que evite imponer un orden arbitrario y garantice que los nodos no sean visitados más de una vez. Esto es esencial para prevenir sesgos en los resultados derivados del orden en que se procesan los nodos.

Con el objetivo de realizar un muestreo aleatorio sin un orden predeterminado, se han desarrollado diversas técnicas y métodos. Estas técnicas surgen de la combinación de dos conceptos previamente introducidos y detallados en el capítulo 2. En primer lugar, generamos un árbol generador mínimo con base en la matriz adyacencia A de la gráfica G , por medio del algoritmo de Kruskal, descrito a continuación:

Algorithm 3 Algoritmo de Kruskal para el Árbol de Expansión Mínima (MST)

Require: Grafo no dirigido $G = (V, E)$ con pesos en las aristas

Ensure: Árbol de expansión mínima (MST)

```

1: Inicializar  $MST \leftarrow \emptyset$  ▷ Conjunto vacío para el MST
2: Ordenar las aristas  $E$  en orden creciente de pesos
3: Inicializar estructura Union-Find para los vértices  $V$ 
4: for cada arista  $(u, v) \in E$  en orden creciente de peso do
5:   if  $\text{FIND}(u) \neq \text{FIND}(v)$  then ▷ Si  $u$  y  $v$  no están en el mismo componente
6:      $MST \leftarrow MST \cup \{(u, v)\}$  ▷ Añadir arista al MST
7:      $\text{UNION}(u, v)$  ▷ Unir los componentes de  $u$  y  $v$ 
8:   end if
9:   if Número de aristas en  $MST = |V| - 1$  then ▷ MST completo
10:    break
11:   end if
12: end for
13: return  $MST$  ▷ Devolver el árbol de expansión mínima

```

Posteriormente a utilizar el algoritmo de Kruskal para generar el árbol generador mínimo, convertimos el árbol resultante a una matriz simétrica de la siguiente forma : $A_s = A_{mst} + A_{mst}^T$. El objetivo de generar el árbol generador mínimo es recorrer la estructura de la gráfica completa sin recorrer todas las aristas, sino sólo las más importantes que en este caso, son las que dan la estructura a la gráfica original. Una vez obtenido el MST, procedemos a recorrerlo mediante un orden impuesto por el algoritmo "DFS", el cual es un algoritmo que sirve para recorrer una gráfica en profundidad, es decir a lo "largo" asegurando así que visitaremos todos los nodos de dicha gráfica para preservar lo que nos interesa que es la estructura de la gráfica. A continuación enunciaremos el algoritmo DFS, y su implementación en nuestro método:

Algorithm 4 DFS Order

Require: Nodo actual *nodo*, lista de nodos visitados *visitados*, lista del orden *orden*
Ensure: Lista *orden* con el orden en que los nodos son visitados

```

1: function DFS_ORDER(nodo, visitados, orden)
2:    $\text{visitados}[\text{nodo}] \leftarrow \text{True}$  ▷ Marcar el nodo como visitado
3:   Agregar nodo a la lista orden ▷ Registrar el nodo en el orden de visita
4:   for all vecino en posiciones donde  $\text{mst}[\text{nodo}] = 1$  do
5:     if  $\text{visitados}[\text{vecino}] = \text{False}$  then
6:       DFS_ORDER(vecino, visitados, orden) ▷ Llamada recursiva para el vecino
       no visitado
7:     end if
8:   end for
9: end function

```

Posterior a establecer el orden en el que se recorrerá la gráfica, generamos un orden entre nodos para visitar los nodos más lejanos del nodo actual, asegurando de esta forma ir lo más

lejos posible en el menor número de pasos.

4.0.4. Muestreo por Esferas

Una de las formas de mantener la estructura original de la gráfica es enfocándonos en su estructura local, es decir, podemos mirar a partir de un nodo v su vecindario, así como también tomar una pequeña esfera $S_v(r)$ con radio r y centro en v que interseque su vecindario $N_G(v)$, para así mantener las propiedades locales y poder asegurar que se preserve la estructura recibida.

Para identificar la muestra resultado de la intersección entre $N_G(V)$ y la esfera $S_v(r)$ se ha construido un método denominado Método polar para desviaciones normales [?, p. 122-123]. A continuación describiremos un algoritmo que sirve como base a la implementación realizada para muestrear uniformemente puntos en el volumen de una esfera, se conoce cómo *Método polar para desviaciones normales* o *Método polar de Marsaglia*.

Algorithm 5 Método polar para desviaciones normales

- 1: **P1** [Obtener variables uniformes]
- 2: Generar dos variables aleatorias independientes $U_1, U_2 \sim U(0, 1)$.
- 3: Asignar $V_1 \leftarrow 2U_1 - 1$, $V_2 \leftarrow 2U_2 - 1$, ahora $V_1, V_2 \sim U(-1, 1)$.
- 4: **P2** [Calcular S]
- 5: Asignar $S \leftarrow V_1^2 + V_2^2$.
- 6: **P3** [*¿Es $S \geq 1$?*]
- 7: **if** $S \geq 1$ **then**
- 8: Regresar al paso **P1**.
- 9: **end if**
- 10: **P4** [Calcular X_1, X_2]
- 11: Asignar:

$$X_1 \leftarrow V_1 \sqrt{\frac{-2 \ln S}{S}}, \quad X_2 \leftarrow V_2 \sqrt{\frac{-2 \ln S}{S}}$$

Para probar la validez del método anterior utilizaremos geometría analítica y un poco de cálculo.

Demostración. Notamos que si $S < 1$ en el paso *P3*, el punto en el plano cartesiano con las coordenadas (V_1, V_2) es un punto aleatorio uniformemente distribuido dentro del círculo unitario.

Transformando a coordenadas polares tenemos que: $V_1 = R \cos \theta$, $V_2 = R \sin \theta$ y sustituyendo tenemos que:

$$S = R^2, \quad X_1 = (R \cos \theta) \sqrt{\frac{-2 \ln S}{R^2}}, \quad X_2 = (R \sin \theta) \sqrt{\frac{-2 \ln S}{R^2}}$$

De aquí observamos que:

$$\begin{aligned}
 X_1 &= (R \cos \theta) \sqrt{\frac{-2 \ln S}{R^2}} \\
 &= R \cos \theta \frac{\sqrt{-2 \ln S}}{\sqrt{R^2}} \\
 &= R \cos \theta \frac{\sqrt{-2 \ln S}}{|R|} \\
 &= \cos \theta \sqrt{-2 \ln S}
 \end{aligned}$$

Pues $R > 0$ De igual forma $X_2 = \sin \theta \sqrt{-2 \ln S}$. Utilizando coordenadas polares y haciendo:

$$R' = \sqrt{-2 \ln S}, \quad \theta' = \theta$$

Tenemos que $X_1 = R' \cos \theta'$, $X_2 = R' \sin \theta'$ Así, observamos que R' y θ' son independientes pues R y θ lo son. También $\theta' \sim U(0, 2\pi)$ pues es el rango de las funciones $\sin$ y $\cos$.

A su vez notamos que:

$$\begin{aligned}
 P(R' \leq r) &= P(-2 \ln S \leq r^2) \\
 &= P(S \geq e^{-\frac{r^2}{2}})
 \end{aligned}$$

Por otro lado, como $S \sim U(0, 1)$ entonces su función de distribución acumulada es :

$$F(s) = P(S \leq s) = s, \quad s \in [0, 1]$$

De esta forma la probabilidad complementaria es:

$$\begin{aligned}
 P(S < s) &= 1 - P(S \geq s) \\
 P(S \geq s) &= 1 - P(S < s)
 \end{aligned}$$

Por tanto:

$$\begin{aligned}
 P(S \geq e^{-\frac{r^2}{2}}) &= 1 - P(S < e^{-\frac{r^2}{2}}) \\
 &= 1 - e^{-\frac{r^2}{2}}
 \end{aligned}$$

Ahora, la probabilidad que R' esté entre r y $r + dr$ es la derivada de $1 - e^{-\frac{r^2}{2}}$ que es igual a $r e^{-\frac{r^2}{2}} dr$. Similarmente la probabilidad de que θ' esté entre θ y $\theta + d\theta$ es $\frac{1}{2\pi} d\theta$ De esta forma, la probabilidad conjunta de $X_1 \leq x_1$ y $X_2 \leq x_2$ es:

$$\begin{aligned}
 \int_{\{(r, \theta) | r \cos \theta \leq x_1, r \sin \theta \leq x_2\}} \frac{1}{2\pi} e^{-\frac{r^2}{2}} r dr d\theta &= \frac{1}{2\pi} \int_{\{(x, y) | x \leq x_1, y \leq x_2\}} e^{-\frac{(x^2 + y^2)}{2}} dx dy \\
 &= \left(\sqrt{\frac{1}{2\pi}} \int_{-\infty}^{x_1} e^{-\frac{x^2}{2}} dx \right) \left(\sqrt{\frac{1}{2\pi}} \int_{-\infty}^{x_2} e^{-\frac{y^2}{2}} dy \right)
 \end{aligned}$$

Lo cual prueba que X_1 y X_2 son variables aleatorias independientes con distribución

normal, tal como se requería.

□

Basado en el algortimo 3, podemos derivar un método que permita generar puntos aleatorios dentro de una esfera en 3 dimensiones. Sean $U_1, U_2 \sim \mathcal{U}(0, 1)$ entonces, hacemos V_1, V_2 y S como:

$$V_1 = 2U_1 - 1, \quad V_2 = 2U_2 - 1, \quad S = V_1^2 + V_2^2$$

Notamos entonces que $V_1, V_2 \sim \mathcal{U}(-1, 1)$

Por otro lado, sabemos que la esfera $\mathbb{S}^2$ tiene las siguientes coordenadas:

$$X = \text{sen}(\theta)\cos(\phi), \quad Y = \text{sen}(\theta)\text{sen}(\phi), \quad Z = \cos(\theta)$$

Notamos que $Z = \cos(\theta)$ tiene un rango de valores de $[-1, 1]$, entonces utilizando la transformación $Z = 2S - 1$ para $S = V_1^2 + V_2^2 \leq 1$ esto implica que. $Z = \cos(\theta) = 2S - 1 \in [-1, 1]$ Por otro lado;

$$\begin{aligned} \cos^2(\theta) + \text{sen}^2(\theta) &= 1 \\ \text{sen}^2(\theta) &= 1 - \cos^2(\theta) \\ \text{sen}^2(\theta) &= 1 - Z^2 \\ \text{sen}(\theta) &= \sqrt{1 - Z^2} \end{aligned}$$

Por tanto:

$$X = \sqrt{1 - Z^2}\cos(\phi), \quad Y = \sqrt{1 - Z^2}\text{sen}(\phi) \quad (1)$$

Por otro lado, notamos que por construcción

$$\begin{aligned} S &= V_1^2 + V_2^2 \\ \sqrt{S} &= \sqrt{V_1^2 + V_2^2} \end{aligned}$$

Y además, en el plano XY

$$\begin{aligned} \cos(\phi) &= \frac{V_1}{\sqrt{V_1^2 + V_2^2}} \\ &= \frac{V_1}{\sqrt{S}} \end{aligned}$$

Análogamente: $\text{sen}(\phi) = \frac{V_2}{\sqrt{S}}$

Sustituyendo en 1

$$\begin{aligned}
X &= \sqrt{1 - Z^2} \frac{V_1}{\sqrt{S}} \\
&= \frac{\sqrt{1 - Z^2}}{\sqrt{S}} V_1 \\
&= \sqrt{\frac{1 - Z^2}{S}} V_1
\end{aligned}$$

Desarrollando $\sqrt{\frac{1 - Z^2}{S}}$:

$$\begin{aligned}
\sqrt{\frac{1 - Z^2}{S}} &= \sqrt{\frac{1 - (2S - 1)^2}{S}} \\
&= \sqrt{\frac{1 - ((2S)^2 + (4S)(-1) + 1)}{S}} \\
&= \sqrt{\frac{1 - (4S^2 - 4S + 1)}{S}} \\
&= \sqrt{\frac{-4S^2 + 4S}{S}} \\
&= \sqrt{\frac{4S(-S + 1)}{S}} \\
&= \sqrt{4(1 - S)} \\
&= \sqrt{4}\sqrt{1 - S} \\
&= 2\sqrt{1 - S}
\end{aligned}$$

Es decir se toma la parte positiva de la raíz cuadrada ya que se están normalizando coordenadas, por otro lado notamos que $1 - S \geq 0$ pues $S \leq 1$. Así, por convención hacemos $\alpha = 2\sqrt{1 - S}$, por tanto $X = \alpha V_1$ y análogamente, $Y = \alpha V_2$. De esta forma la generación de puntos aleatorios en la superficie de la esfera es:

$$(\alpha V_1, \alpha V_2, 2S - 1)$$

Donde:

$$\begin{aligned}
X^2 + Y^2 + Z^2 &= (\alpha V_1)^2 + (\alpha V_2)^2 + (2S - 1)^2 \\
&= \alpha^2(V_1^2 + V_2^2) + (2S - 1)^2 \\
&= S(2\sqrt{1 - S})^2 + (2S - 1)^2 \\
&= 4S(1 - S) + 4S^2 - 4S + 1 \\
&= -4S^2 + 4S + 4S^2 - 4S + 1 \\
&= 1
\end{aligned}$$

Por último, para generar puntos aleatorios, dentro del volumen de la esfera, basta con generar un radio aleatorio de la siguiente forma: Dado $R = \mathcal{U}(0, 1)^{\frac{1}{3}}$, los puntos distribuidos

dentro de la esfera están dados por:

$$R(\alpha V_1, \alpha V_2, 2S - 1)$$

De esta forma mostraremos el algoritmo resultante en la siguiente forma;

Algorithm 6 Generar Puntos Uniformes en una Esfera

Require: Coordenadas del centro h, k, l , número de puntos num_puntos

Ensure: Matriz de puntos uniformemente distribuidos en la esfera

```

1: function GENERARPUNTOSUNIFORMES( $h, k, l, num\_puntos$ )
2:   Inicializar una lista vacía  $puntos$ 
3:   for  $i \leftarrow 1$  to  $num\_puntos$  do
4:     repeat
5:       Generar  $V1, V2 \leftarrow \text{UNIFORM}(-1, 1)$ 
6:       Calcular  $S \leftarrow V1^2 + V2^2$ 
7:     until  $S < 1$ 
8:     Calcular  $a \leftarrow \sqrt{1 - S}$ 
9:     Calcular  $x \leftarrow a \cdot V1 + h$ 
10:    Calcular  $y \leftarrow a \cdot V2 + k$ 
11:    Calcular  $z \leftarrow 2 \cdot S - 1 + l$ 
12:    Generar  $r \leftarrow \text{UNIFORM}(0, 1)^{\frac{1}{3}}$ 
13:    Agregar  $[r \cdot x, r \cdot y, r \cdot z]$  a la lista  $puntos$ 
14:  end for
15:  return Matriz convertida a partir de  $puntos$ 
16: end function

```

4.0.5. Reasignación de Adyacencias para Preservar la Estructura

En el punto anterior notamos que cuando se elimina un nodo contenido dentro de la esfera de muestreo, los vecinos de este nodo pierden cierta información estructural contenida en el nodo eliminado, para evitar esto y preservar la estructura lo más parecida posible a la inicial hacemos una reasignación de adyacencias.

Describiremos a continuación el proceso para eliminar un nodo v de G y que sus vecinos mantengan la adyacencia entre ellos, es decir si dos nodos u, w eran vecinos de v entonces después de eliminar v , u y w pasaran a ser vecinos entre ellos.

Definición 67. Sea G una gráfica y A su matriz de adyacencia, entonces definimos al elemento $(A)_{ij}$ como el elemento ij -ésimo de la matriz A , donde $i, j \in \{0, \dots, \nu_G - 1\}$

Ejemplo 17. Sea la matriz A dada por :

$$A(G) = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix} \quad (4.1)$$

Entonces los elementos $(A)_{02}, (A)_{23}, (A)_{54}$ son iguales a :

$$(A)_{02} = 0$$

$$(A)_{23} = 1$$

$$(A)_{54} = 1$$

Dada esta definición podemos ahora establecer una operación que sirve como punto de partida para implementar un método de eliminación controlada para los nodos de la gráfica. La siguiente definición implementará una operación entre los elementos de la matriz de adyacencia A_n donde n representa el paso antes de realizar la operación y por ende la matriz A_{n+1} denota a la matriz de adyacencia A después de la operación.

Definición 68 (Eliminación del nodo k). Sea G una gráfica A_n la matriz de adyacencia en el paso n , A_{n+1} la matriz de adyacencia en el paso $n + 1$, k el nodo a eliminar y $N_G(k)$ el vecindario del nodo k .

Entonces definimos la función,

$$\psi : (A_n)_{ij} \times (A_n)_{kj} \rightarrow (A_{n+1})_{ij}$$

Dada por:

$$\psi_{ij} := (A_{n+1})_{ij} = \begin{cases} (A_n)_{ij} +_{or} (A_n)_{kj} & , \quad i \neq j, j \neq k \\ 0 & , \quad k = j, i = j \end{cases} \quad (4.2)$$

Con $i \in N_G(k) \cup \{k\}$, además $\psi_{kj} = 0, \forall j \in \{0, \dots, \nu_G\}$

Ejemplo 18. Dada la matriz $A_0 = A(G)$ del ejemplo (2), $k = 5$, $\nu_G = 6$, $j \in \{0, \dots, 5\}$ entonces calcularemos los valores de ψ .

Observamos que $N_G(k) = \{1, 2, 4\}$, entonces:

$$\begin{aligned}
\psi_{10} &:= (A_1)_{10} = (A_0)_{10} +_{or} (A_0)_{50} \\
&= 1 +_{or} 0 \\
&= 1 \\
\psi_{11} &:= (A_1)_{11} = 0 \\
\psi_{12} &:= (A_1)_{12} = (A_0)_{12} +_{or} (A_0)_{52} \\
&= 1 +_{or} 1 \\
&= 1 \\
\psi_{5j} &:= (A_1)_{5j} = 0 \quad \forall j \in \{0, \dots, 5\}
\end{aligned}$$

Y notamos que análogamente para $i = \{2, 4\}$. De esta forma la matriz A_1 quedaría de la siguiente forma:

$$A_1(G) = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad (4.3)$$

Así, hemos definido una operación para las matrices de adyacencia donde, dado un nodo k podemos eliminarlo de dicha matriz, es decir, $(A)_{kj}, (A)_{ik} = 0$, donde $i, j \in \{0, \dots, \nu_G\}$ asegurando que sus vecinos preserven la adyacencia entre ellos, sin modificar la estructura original de la gráfica.

De esta forma podemos definimos el algoritmo de la siguiente manera:

Algorithm 7 Eliminar Nodo con Redistribución de Adyacencias

Require: Nodo a eliminar $nodo$, matriz de adyacencia $matrizAdj$

Ensure: Matriz de adyacencia actualizada sin el nodo y con conexiones redistribuidas

```

1: function ELIMINANODOCONADYACENCIA( $nodo, matrizAdj$ )
2:   Obtener los vecinos del nodo:  $neighbors \leftarrow \text{WHERE}(matrizAdj[nodo] = 1)$   $\triangleright$  Lista
   de nodos directamente conectados a  $nodo$ .
3:   for all  $v$  en  $neighbors$  do
4:      $matrizAdj[v] \leftarrow \text{LOGICALOR}(matrizAdj[v], matrizAdj[nodo])$   $\triangleright$  Redistribuir
     conexiones del nodo eliminado a sus vecinos.
5:   end for
6:    $matrizAdj[nodo, :] \leftarrow 0$   $\triangleright$  Eliminar todas las conexiones salientes del nodo.
7:    $matrizAdj[:, nodo] \leftarrow 0$   $\triangleright$  Eliminar todas las conexiones entrantes al nodo.
8:   return  $matrizAdj$ 
9: end function

```

4.0.6. Método general para la reducción de nodos

Una vez establecidos los métodos descritos anteriormente, es posible combinar sus principios para desarrollar un enfoque generalizado de reducción de nodos.

En esta sección, exploraremos en detalle dicho enfoque, examinando su complejidad computacional, describiendo su implementación paso a paso y destacando las ventajas clave que se derivan de su aplicación.

El proceso de muestreo eficiente en una gráfica se basa en reducir el número de nodos mientras se conservan sus propiedades clave. Primero, se genera un orden de recorrido utilizando un árbol generador mínimo y el algoritmo DFS, lo que permite recorrer toda la gráfica y establecer un orden basado en DFS.

En el ciclo principal, se iteran los nodos hasta alcanzar el tamaño deseado de la muestra. Se calculan las distancias promedio entre los nodos ordenados y el centroide de la muestra, priorizando los más lejanos para evitar sesgos. Con el nuevo orden, se generan puntos alrededor de cada nodo utilizando una esfera, identificando y eliminando vecinos dentro de un radio definido, manteniendo la adyacencia. Este proceso se repite hasta obtener la muestra requerida. El algoritmo se divide en fases para facilitar su comprensión.

Algorithm 8 Método General

Require: Lista de nodos con coordenadas x , Matriz de adyacencia $adjacency$, Tamaño de muestra $size$, Radio $alcance$, Tamaño de la muestra de esfera $esfera_size$

Ensure: Nueva lista de nodos new_x , Matriz de adyacencia Actualizada new_adj

```

1: function SAMPLING( $x, adjacency, size, alcance, esfera\_size$ )
2:   Obtener el árbol generador mínimo (MST) desde  $adjacency$  ▷ Algoritmo 3
3:   Inicializar  $visited[] \leftarrow FALSE$  ;  $order[] \leftarrow empty\_list$ 
4:   Obtener orden mediante DFS  $dfs\_order(0, visited, order)$  ▷ Algoritmo 4
5:   Inicializar  $removed\_nodes[] \leftarrow empty\_list$ ;  $queue[] \leftarrow copy(order)$ 
6:   while  $number\_nodes - removed\_nodes \geq size$  do
7:     if  $removed\_nodes$  no es vacío then
8:        $dist \leftarrow dist(current\_node, centroid)$ 
9:       Ordena  $queue$  por distancias en orden descendente
10:    end if
11:    Obtener coordenadas del primer elemento de  $queue$ .
12:     $idx \leftarrow queue.pop()$ ;  $h, k, l \leftarrow x[idx]$ 
13:    Obtenemos esfera uniforme GENERAPUNTOSUNIFORMES( $idx$ ) ▷ Algoritmo 6
14:    Hacemos  $neighbors \leftarrow neighbors \cap GeneraPuntosUniformes(idx)$ 
15:    for all  $vecinos \in vecinos - eliminados$  do
16:      ELIMINANODOCONADYACENCIA( $n, adjacency$ ) ▷ Algoritmo 7
17:    end for
18:  end while
19:  return  $new\_x, new\_adj$ 
20: end function

```

4.0.7. Análisis de Complejidad

El análisis de complejidad de nuestro algoritmo se realiza descomponiendo sus principales componentes y justificando cada cálculo. La construcción del Árbol Generador Mínimo (MST) utiliza el algoritmo de Kruskal, que tiene una complejidad de $\mathcal{O}(E \log(V))$ [?], donde E es el número de aristas y V el número de vértices de la gráfica. Este paso es esencial para establecer una estructura inicial que guía el muestreo de nodos. El siguiente paso es ordenar los nodos del grafo mediante un recorrido en profundidad (DFS), cuya complejidad es $\mathcal{O}(V + E)$ [?]. Esto se debe a que el DFS visita cada vértice y cada arista exactamente una vez. Este orden nos permite recorrer sistemáticamente los nodos y controlar las operaciones de muestreo.

El bucle principal del algoritmo, que se ejecuta hasta alcanzar el tamaño deseado de la muestra, domina la mayor parte del costo computacional, en el peor de los casos este bucle se ejecuta V veces, considerando que el tamaño de la muestra puede representar una cota ya que en cada iteración se elimina un nodo por tanto su costo podría acotarse por $\mathcal{O}(V)$.

Dentro de este bucle, el cálculo de distancias entre los nodos y el centroide de la muestra se calcula mediante la norma euclideana, es decir se calcula la distancia a lo más V veces y se repite por el número d de dimensión que en este caso $d = 3$ por tanto la complejidad total es: $\mathcal{O}(V)$.

Luego, la complejidad del algoritmo para generar la muestra uniforme está determinado por el número de veces que se ejecuta su bucle principal, ya que, las operaciones realizadas no dependen del número de puntos a generar, por lo tanto la complejidad es constante, de esta forma el número de pasos que realiza el método es en el orde de *esfera.size* que, para términos de la implementación de dicho algoritmo, siempre es constante.

La intersección de los vecinos generados en la esfera con los vecinos del nodo en la matriz de adyacencia tiene un costo lineal de $\mathcal{O}(V)$. Este cálculo depende del tamaño de los conjuntos comparados, donde el conjunto mayor está determinado por el número de vértices de la gráfica. Una vez identificados los nodos dentro de la esfera, el algoritmo actualiza la matriz de adyacencia. Dado que esta matriz tiene un tamaño de $V \times V$, las operaciones de reajuste tienen un costo de $\mathcal{O}(V^2)$.

Finalmente, los nodos eliminados deben ser descartados de la matriz de adyacencia, lo que implica un costo adicional de $\mathcal{O}(V)$ por el número de pasos k , donde k es el número de nodos eliminados que para terminos de la implementación lo podemos dejar como una constante, por lo tanto la complejidad final sería $\mathcal{O}(V)$. Este costo se suma al procesamiento necesario para mantener la consistencia de la estructura de datos. Resumiendo, notamos que las operaciones más costosas que otorgan la complejidad final al algoritmo es la obtención del MST $\mathcal{O}(E \log V)$ y las iteraciones del bucle principal, que resulta ser $\mathcal{O}(V * V^2) = \mathcal{O}(V^3)$ pues, las operaciones se ejecutan V veces multiplicado por el reordenamiento de la matriz de adyacencia, entonces dado las dos operaciones mas costosas en diferentes fases del algoritmo la complejidad final está dada por: $\mathcal{O}(E \log V + V^3)$.

Como observación final, en el peor de los casos sabemos que $E = V^2$, pero como estamos trabajando con un tipo de gráficas especiales, para este caso $E \neq V^2$, en específico $E < V^2$ captura equivarianzas sobre la superficie por eso los resultados parecen ser mejores.

Capítulo 5

Resultados y conclusiones

Las arquitecturas comúnmente utilizadas en el procesamiento de imágenes requieren que los datos se encuentren ordenados en cierto formato, el cual puede ser en cuadrículas 3D, lo que permite que el proceso de distribución de peso se realice para optimizar el núcleo. Sin embargo, este enfoque puede resultar altamente costoso, ya que maneja una gran cantidad de parámetros que, en muchos casos, no aportan información relevante a la red y generan un desperdicio computacional significativo. Para evitar estos inconvenientes, surge una arquitectura denominada PointNet, diseñada específicamente para procesar directamente nubes de puntos 3D sin la necesidad de transformarlas a una estructura regular como cuadrículas o vóxeles. PointNet aprovecha la naturaleza no estructurada de las nubes de puntos y utiliza operaciones simétricas, como el max pooling, para garantizar la invariancia al orden de los puntos. De este modo, permite realizar el procesamiento de datos 3D de manera más eficiente, capturando tanto características globales como locales directamente de los puntos de entrada, lo que simplifica y optimiza el análisis de formas y estructuras tridimensionales. [?] En el presente capítulo analizaremos la forma en que el algoritmo presentado en el capítulo anterior influye en los resultados de aplicar la arquitectura PointNet para la clasificación de nudos de Lissajous basado en la característica de Euler. La descripción de la arquitectura está basada en el artículo [?].

5.0.1. Arquitectura PointNet

La arquitectura, está diseñada de tal forma que recibe un conjunto de puntos sin orden en 3 dimensiones, es decir, $\{P_i \mid i = 1, \dots, n\}$ donde cada punto P_i es un vector con coordenadas (x_i, y_i, z_i) , es decir, una matriz $X \in \mathbb{R}^{n \times 3}$ la cual evita la pérdida de información y reduce la complejidad computacional, a diferencia de otros algoritmos que convierten las nubes de puntos en imágenes 2D.

La entrada de la red se basa en tres propiedades principales de los puntos en el espacio Euclideo:

1. **Sin orden:** La nube de puntos es un conjunto de puntos sin un orden en específico es decir la red consume N puntos en 3d que son invariantes a $N!$ permutaciones en el conjunto de datos de entrada.
2. **Interacción entre puntos:** Los puntos provienen de un espacio métrico, lo cual

otorga un subconjunto significativo de características por lo cual el modelo de poder capturar estructuras locales entre puntos cercanos.

3. **Invariancia bajo transformaciones:** La nube de puntos es invariante bajo ciertas transformaciones por ejemplo, rotar y trasladar el conjunto de puntos no debería modificar la estructura global de dicho conjunto.

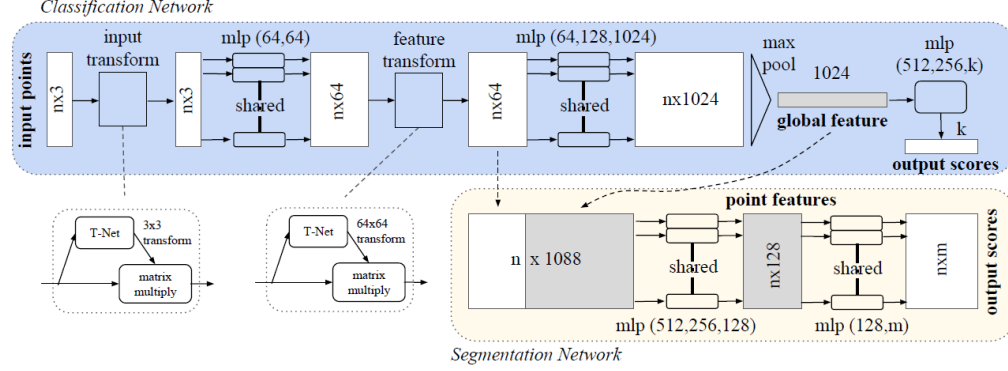


Figura 5.1: Arquitectura PointNet [?]

En la medida en que el modelo debe permanecer invariante bajo permutaciones de los puntos de entrada, PointNet utiliza una función simétrica para agregar características de puntos individuales. Esta función simétrica es típicamente una operación de max pooling, que selecciona el valor máximo a lo largo de una dimensión específica. La red se compone de varias capas de convolución 1D que operan sobre cada punto individualmente, seguidas de una capa de max pooling global que agrega las características extraídas de todos los puntos, pero principalmente sus capas siguen la siguiente ecuación que rige el funcionamiento y la validez de PointNet como aproximador universal de funciones continuas.

Dado un conjunto de puntos sin orden $\{x_1, x_2, \dots, x_n\}$ con $x_i \in \mathbb{R}^d$ podemos definir un conjunto de funciones que mapean el conjunto de vértices a un vector de la forma:

$$\mathbf{x}'_i = \gamma_{\Theta} \left(\max_{j \in \mathcal{N}(i) \cup \{i\}} \mathbf{h}_{\Theta}(\mathbf{x}_j, p_j - p_i) \right)$$

Donde podemos recalcar, que cada punto se actualiza en función de la información proporcionada por sus vecinos $j \in \mathcal{N}(i)$. De aquí que podemos notar que la función $\mathbf{h}_{\Theta}(\mathbf{x}_j, p_j - p_i)$ calcula la interacción entre el punto i y sus vecinos lo que puede considerarse como el mensaje que el punto i recibe. Además la operación $\max$ indica que para cada punto i se toma el valor máximo de las interacciones de todos sus vecinos y de sí mismo actuando como un operador de agregación. Por último notamos que la función γ_{θ} aplica una transformación no lineal a las representaciones agregadas, por lo tanto podríamos tomarlo como una función de actualización. De esta forma podemos ver que la ecuación anterior la podríamos moldear como una GNN.

La arquitectura de PointNet se puede dividir en tres módulos principales:

1. **Transformación de entrada:** PointNet aprende una transformación afín para alinear la nube de puntos de manera canónica, lo cual se logra mediante una subred llamada

T-Net la cual predice una matriz de transformación $T \in \mathbf{R}^{3 \times 3}$, es decir

$$X^* = X \cdot T$$

, donde X^* es la nube de puntos alineada y T es la matriz predicha por **T-Net**

2. **Extracción de características locales y globales:** A través de una red MLP, se extraen características locales de cada punto, es decir, para cada punto $x_i \in \mathbf{R}^3$, el MLP lo mapea a un espacio de características de mayor dimensión:

$$h_i = MLP(x_i), \quad h_i \in \mathbf{R}^d$$

lo cual, al mapearse en todos los puntos, resulta en una matriz de características $H \in \mathbf{R}^{n \times d}$. Similar a la transformación de entrada, la red utiliza otra **T-Net** para aprender una transformación afín en el espacio de características, es decir:

$$H^* = H \cdot T_f$$

donde $T_f \in \mathbf{R}^{d \times d}$ es la matriz de transformación aprendida. Luego, una capa de max pooling global agrega estas características en un vector de características globales $g \in \mathbf{R}^d$, es decir

$$g_j = \max_{i=1, \dots, n} H_{ij} \quad \forall j = 1, \dots, d$$

3. **Clasificación o segmentación:** Dependiendo de la tarea, el vector de características globales se puede utilizar para la clasificación de la nube de puntos completa, o se pueden concatenar características locales y globales para realizar segmentación punto a punto. En el caso de la clasificación, podemos utilizar una red MLP para predecir la clase, es decir

$$y = MLP(g)$$

donde $y \in \mathbf{R}^k$ con k clases y para el vector global g .

5.0.2. Resultados de la evaluación

Ahora que describimos el funcionamiento y arquitectura de los modelos PointNet, presentaremos a continuación los resultados obtenidos de aplicar esta arquitectura combinado de el método de muestreo contra la arquitectura aplicada sin el método de muestreo. En esta sección, también, se justifica el desempeño observado considerando las diferencias en la representación de datos y el procesamiento de la información.

Podemos observar una diferencia notoria en los resultados presentados, en primer lugar podemos observar que en el modelo con muestreo obtenemos métricas de precisión, recall y F1-score considerablemente más altos en la mayoría de las clases lo cual nos indica que este modelo clasifica correctamente las clases determinadas, también con base en el recall podemos decir que el modelo detecta en la mayoría los casos positivos y por último la F1-score nos indica que la reducción de nodos ayudó a mantener la información clave sin perder la representatividad en las clases.

	precision	recall	f1-score	support
0	0.85	0.85	0.85	20
1	0.77	0.50	0.61	20
2	0.95	0.90	0.92	20
3	0.83	0.95	0.88	20
4	0.80	0.80	0.80	20
5	1.00	0.30	0.46	20
6	0.57	1.00	0.73	20
7	0.72	0.90	0.80	20
8	0.84	0.80	0.82	20
9	0.90	0.90	0.90	20
accuracy			0.79	200
macro avg	0.82	0.79	0.78	200
weighted avg	0.82	0.79	0.78	200

Figura 5.2: Resultados obtenidos aplicando el muestreo general

	precision	recall	f1-score	support
0	0.32	1.00	0.49	20
1	0.29	0.35	0.32	20
2	0.58	0.95	0.72	20
3	1.00	0.30	0.46	20
4	0.00	0.00	0.00	20
5	0.42	0.55	0.48	20
6	0.00	0.00	0.00	20
7	1.00	0.25	0.40	20
8	0.61	0.85	0.71	20
9	0.81	0.65	0.72	20
accuracy			0.49	200
macro avg	0.50	0.49	0.43	200
weighted avg	0.50	0.49	0.43	200

Figura 5.3: Resultados obtenidos sin aplicar el muestreo general

5.0.3. Conclusiones

Apéndice A

Definiciones básicas

Definición 69. (*Espacio métrico*) Un espacio métrico consta de un conjunto X y una función $d : X \times X \rightarrow \mathbb{R}^+$, llamada métrica, o distancia que satisface los siguientes axiomas:

1. $d(x, y) = 0$ si y sólo si $x = y$.
2. $d(x, y) = d(y, x)$ para todo $x, y \in X$.
3. $d(x, y) \leq d(x, z) + d(z, y)$ para todo $x, y, z \in X$

Definición 70. (*Bola abierta*) Sea $x \in X$ con X espacio métrico, $\epsilon \in \mathbb{R}^+$. Definimos la bola abierta con centro en x y radio ϵ como:

$$\mathcal{B}_\epsilon(x) = \{y \in X \mid d(x, y) < \epsilon\}$$

Definición 71. (*Vecindad*) Sea X un espacio métrico y U un subconjunto de este, decimos que $U \subseteq X$ es una **vecindad** de x si existe $\epsilon > 0$ tal que $\mathcal{B}_\epsilon(x) \subseteq U$

Definición 72. (*Conjunto abierto*) Sea X un espacio métrico. Decimos que un subconjunto A de X es **abierto** si A es vecindad de x para toda $x \in A$. Es decir $A \subseteq X$ es abierto si y sólo si para todo $x \in A$ existe $\epsilon > 0$ tal que $\mathcal{B}_\epsilon(x) \subseteq A$

Teorema 7. (*Unión e intersección de abiertos*) Sea $\mathcal{A}$ el conjunto de todos los abiertos en un espacio métrico X . Entonces se cumplen las siguientes afirmaciones:

1. Sea $\mathcal{I}$ un conjunto arbitrario de índices. Si $\{A_i\}_{i \in \mathcal{I}}$ es una familia de elementos en $\mathcal{A}$, entonces $\bigcup_{i \in \mathcal{I}} A_i$ es elemento de $\mathcal{A}$
2. Sea $\mathcal{I}$ un conjunto finito de índices. Si $\{A_i\}_{i \in \mathcal{I}}$ es una familia de elementos en $\mathcal{A}$, entonces $\bigcap_{i \in \mathcal{I}} A_i$ es elemento de $\mathcal{A}$